

# Introduction

Welcome to The Rust Edition Guide! "Editions" are Rust's way of introducing changes into the language that would not otherwise be backwards compatible.

In this guide, we'll discuss:

- What editions are
- Which changes are contained in each edition
- How to migrate your code from one edition to another

# What are Editions?

In May 2015, the [release of Rust 1.0](#) established "[stability without stagnation](#)" as a core Rust axiom. Since then, Rust has committed to a pivotal rule: once a feature is [released through stable](#), contributors will continue to support that feature for all future releases.

However, there are times when it's useful to make backwards-incompatible changes to the language. A common example is the introduction of a new keyword. For instance, early versions of Rust didn't feature the `async` and `await` keywords.

If Rust had suddenly introduced these new keywords, some code would have broken: `let async = 1;` would no longer work.

Rust uses **editions** to solve this problem. When there are backwards-incompatible changes, they are pushed into the next edition. Since editions are opt-in, existing crates won't use the changes unless they explicitly migrate into the new edition. For example, the latest version of Rust doesn't treat `async` as a keyword unless edition 2018 or later is chosen.

Each crate chooses its edition [within its Cargo.toml file](#). When creating a new crate with Cargo, it will automatically select the newest stable edition.

## Editions do not split the ecosystem

When creating editions, there is one most consequential rule: crates in one edition **must** seamlessly interoperate with those compiled with other editions.

In other words, each crate can decide when to migrate to a new edition independently. This decision is 'private' - it won't affect other crates in the ecosystem.

For Rust, this required compatibility implies some limits on the kinds of changes that can be featured in an edition. As a result, changes found in new Rust editions tend to be 'skin deep'. All Rust code - regardless of edition - will ultimately compile down to the same internal representation within the compiler.

## Edition migration is easy and largely automated

Rust aims to make upgrading to a new edition an easy process. When a new edition releases, crate authors may use [automatic migration tooling within cargo](#) to migrate. Cargo will then make minor changes to the code to make it compatible with the new version.

For example, when migrating to Rust 2018, anything named `async` will now use the equivalent [raw identifier syntax](#): `r#async`.

Cargo's automatic migrations aren't perfect: there may still be corner cases where manual changes are required. It aims to avoid changes to semantics that could affect the correctness or performance of the code.

## What this guide covers

In addition to tooling, this Rust Edition Guide also covers the changes that are part of each edition. It describes each change and links to additional details, if available. It also covers corner cases or tricky details crate authors should be aware of.

Crate authors should find:

- An overview of editions
- A migration guide for specific editions
- A quick troubleshooting reference when automated tooling isn't working.

# Creating a new project

A new project created with Cargo is configured to use the latest edition by default:

```
$ cargo new foo
    Creating binary (application) `foo` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-
lang.org/cargo/reference/manifest.html
$ cat foo/Cargo.toml
[package]
name = "foo"
version = "0.1.0"
edition = "2024"

[dependencies]
```

That `edition = "2024"` setting configures your package to be built using the Rust 2024 edition. No further configuration needed!

You can use the `--edition <YEAR>` option of `cargo new` to create the project using some specific edition. For example, creating a new project to use the Rust 2018 edition could be done like this:

```
$ cargo new --edition 2018 foo
    Creating binary (application) `foo` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-
lang.org/cargo/reference/manifest.html
$ cat foo/Cargo.toml
[package]
name = "foo"
version = "0.1.0"
edition = "2018"

[dependencies]
```

Don't worry about accidentally using an invalid year for the edition; the `cargo new` invocation will not accept an invalid edition year value:

```
$ cargo new --edition 2019 foo
error: invalid value '2019' for '--edition <YEAR>'
  [possible values: 2015, 2018, 2021, 2024]

  tip: a similar value exists: '2021'

For more information, try '--help'.
```

You can change the value of the `edition` key by simply editing the `Cargo.toml` file. For example, to cause your package to be built using the Rust 2015 edition, you would set the key as in the following example:

```
[package]
name = "foo"
version = "0.1.0"
edition = "2015"

[dependencies]
```

# Transitioning an existing project to a new edition

Rust includes tooling to automatically transition a project from one edition to the next. It will update your source code so that it is compatible with the next edition. Briefly, the steps to update to the next edition are:

1. Run `cargo update` to update your dependencies to the latest versions.
2. Run `cargo fix --edition`
3. Edit `Cargo.toml` and set the `edition` field to the next edition, for example `edition = "2024"`
4. Run `cargo build` or `cargo test` to verify the fixes worked.
5. Run `cargo fmt` to reformat your project.

The following sections dig into the details of these steps, and some of the issues you may encounter along the way.

It's our intention that the migration to new editions is as smooth an experience as possible. If it's difficult for you to upgrade to the latest edition, we consider that a bug. If you run into problems with this process, please [file a bug report](#). Thank you!

## Starting the migration

As an example, let's take a look at transitioning from the 2015 edition to the 2018 edition. The steps are essentially the same when transitioning to other editions like 2021.

Imagine we have a crate that has this code in `src/lib.rs`:

```
trait Foo {  
    fn foo(&self, i32);  
}
```

This code uses an anonymous parameter, that `i32`. This is [not supported in Rust 2018](#), and so this would fail to compile. Let's get this code up to date!

## Updating your dependencies

Before we get started, it is recommended to update your dependencies. Some dependencies, particularly some proc-macros or dependencies that do build-time code generation, may have compatibility issues with newer editions. New releases may have been made since you last updated which may fix these issues. Run the following:

```
cargo update
```

After updating, you may want to run your tests to verify everything is working. If you are using a source control tool such as `git`, you may want to commit these changes separately to keep a logical separation of commits.

## Updating your code to be compatible with the new edition

Your code may or may not use features that are incompatible with the new edition. In order to help transition to the next edition, Cargo includes the `cargo fix` subcommand to automatically update your source code. To start, let's run it:

```
cargo fix --edition
```

This will check your code, and automatically fix any issues that it can. Let's look at `src/lib.rs` again:

```
trait Foo {  
    fn foo(&self, _: i32);  
}
```

It's re-written our code to introduce a parameter name for that `i32` value. In this case, since it had no name, `cargo fix` will replace it with `_`, which is conventional for unused variables.

`cargo fix` can't always fix your code automatically. If `cargo fix` can't fix something, it will print the warning that it cannot fix to the console. If you see one of these warnings, you'll have to update your code manually. See the [Advanced migration strategies](#) chapter for more on working with the migration process, and read the chapters in this guide

which explain which changes are needed. If you have problems, please seek help at the [user's forums](#).

## Enabling the new edition to use new features

In order to use some new features, you must explicitly opt in to the new edition. Once you're ready to continue, change your `Cargo.toml` to add the new `edition` key/value pair. For example:

```
[package]
name = "foo"
version = "0.1.0"
edition = "2018"
```

If there's no `edition` key, Cargo will default to Rust 2015. But in this case, we've chosen `2018`, and so our code will compile with Rust 2018!

## Testing your code in the new edition

The next step is to test your project on the new edition. Run your project tests to verify that everything still works, such as running `cargo test`. If new warnings are issued, you may want to consider running `cargo fix` again (without the `--edition` flag) to apply any suggestions given by the compiler.

At this point, you may still need to do some manual changes. For example, the automatic migration does not update doctests, and build-time code generation or macros may need manual updating. See the [advanced migrations chapter](#) for more information.

Congrats! Your code is now valid in both Rust 2015 and Rust 2018!

## Reformatting with rustfmt

If you use `rustfmt` to automatically maintain formatting within your project, then you should consider reformatting using the new formatting rules of the new edition.



Before reformatting, if you are using a source control tool such as `git`, you may want to commit all the changes you have made up to this point before taking this step. It can be useful to put formatting changes in a separate commit, because then you can see which changes are just formatting versus other code changes, and also possibly ignore the formatting changes in `git blame`.

```
cargo fmt
```

See the [style editions chapter](#) for more information.

## Migrating to an unstable edition

After an edition is released, there is roughly a three year window before the next edition. During that window, new features may be added to the next edition, which will only be available on the [nightly channel](#). If you want to help test those new features before they are stabilized, you can use the nightly channel to try them out.

The steps are roughly similar to the stable channel:

1. Install the most recent nightly: `rustup update nightly`.
2. Run `cargo +nightly fix --edition`.
3. Edit `Cargo.toml` and place `cargo-features = ["edition20xx"]` at the top (above `[package]`), and change the edition field to say `edition = "20xx"` where `20xx` is the edition you are upgrading to.
4. Run `cargo +nightly check` to verify it now works in the new edition.

**⚠ Caution:** Features implemented in the next edition may not have automatic migrations implemented with `cargo fix`, and the features themselves may not be finished. When possible, this guide should contain information about which features are implemented on nightly along with more information about their status. A few months before the edition is stabilized, all of the new features should be fully implemented, and the [Rust Blog](#) will announce a call for testing.

# Advanced migration strategies

## How migrations work

`cargo fix --edition` works by running the equivalent of `cargo check` on your project with special `lints` enabled which will detect code that may not compile in the next edition. These lints include instructions on how to modify the code to make it compatible on both the current and the next edition. `cargo fix` applies these changes to the source code, and then runs `cargo check` again to verify that the fixes work. If the fixes fail, then it will back out the changes and display a warning.

Changing the code to be simultaneously compatible with both the current and next edition makes it easier to incrementally migrate the code. If the automated migration does not completely succeed, or requires manual help, you can iterate while staying on the original edition before changing `Cargo.toml` to use the next edition.

The lints that `cargo fix --edition` apply are part of a `lint group`. For example, when migrating from 2018 to 2021, Cargo uses the `rust-2021-compatibility` group of lints to fix the code. Check the [Partial migration](#) section below for tips on using individual lints to help with migration.

`cargo fix` may run `cargo check` multiple times. For example, after applying one set of fixes, this may trigger new warnings which require further fixes. Cargo repeats this until no new warnings are generated.

## Migrating multiple configurations

`cargo fix` can only work with a single configuration at a time. If you use [Cargo features](#) or [conditional compilation](#), then you may need to run `cargo fix` multiple times with different flags.

For example, if you have code that uses `#[cfg]` attributes to include different code for different platforms, you may need to run `cargo fix` with the `--target` option to fix for different targets. This may require moving your code between machines if you don't have cross-compiling available.

Similarly, if you have conditions on Cargo features, like `#[cfg(feature = "my-optional-thing")]`, it is recommended to use the `--all-features` flag to allow `cargo fix` to migrate all the code behind those feature gates. If you want to migrate feature code individually, you can use the `--features` flag to migrate one at a time.

## Migrating a large project or workspace

You can migrate a large project incrementally to make the process easier if you run into problems.

In a [Cargo workspace](#), each package defines its own edition, so the process naturally involves migrating one package at a time.

Within a [Cargo package](#), you can either migrate the entire package at once, or migrate individual [Cargo targets](#) one at a time. For example, if you have multiple binaries, tests, and examples, you can use specific target selection flags with `cargo fix --edition` to migrate just that one target. By default, `cargo fix` uses `--all-targets`.

For even more advanced cases, you can specify the edition for each individual target in `Cargo.toml` like this:

```
[[bin]]
name = "my-binary"
edition = "2018"
```

This usually should not be required, but is an option if you have a lot of targets and are having difficulty migrating them all together.

## Partial migration with broken code

Sometimes the fixes suggested by the compiler may fail to work. When this happens, Cargo will report a warning indicating what happened and what the error was. However, by default it will automatically back out the changes it made. It can be helpful to keep the code in the broken state and manually resolve the issue. Some of the fixes may have been correct, and the broken fix may be *mostly* correct, but just need minor tweaking.

In this situation, use the `--broken-code` option with `cargo fix` to tell Cargo not to back

out the changes. Then, you can go manually inspect the error and investigate what is needed to fix it.

Another option to incrementally migrate a project is to apply individual fixes separately, one at a time. You can do this by adding the individual lints as warnings, and then either running `cargo fix` (without the `--edition` flag) or using your editor or IDE to apply its suggestions if it supports "Quick Fixes".

For example, the 2018 edition uses the `keyword-idents` lint to fix any conflicting keywords. You can add `#![warn(keyword_idents)]` to the top of each crate (like at the top of `src/lib.rs` or `src/main.rs`). Then, running `cargo fix` will apply just the suggestions for that lint.

You can see the list of lints enabled for each edition in the [lint group](#) page, or run the `rustc -Whelp` command.

## Migrating macros

Some macros may require manual work to fix them for the next edition. For example, `cargo fix --edition` may not be able to automatically fix a macro that generates syntax that does not work in the next edition.

This may be a problem for both `proc macros` and `macro_rules`-style macros. `macro_rules` macros can sometimes be automatically updated if the macro is used within the same crate, but there are several situations where it cannot. Proc macros in general cannot be automatically fixed at all.

For example, if we migrate a crate containing this (contrived) macro `foo` from 2015 to 2018, `foo` would not be automatically fixed.

```
#[macro_export]
macro_rules! foo {
    () => {
        let dyn = 1;
        println!("it is {}", dyn);
    };
}
```

When this macro is defined in a 2015 crate, it can be used from a crate of any other edition due to macro hygiene (discussed below). In 2015, `dyn` is a normal identifier and

can be used without restriction.

However, in 2018, `dyn` is no longer a valid identifier. When using `cargo fix --edition` to migrate to 2018, Cargo won't display any warnings or errors at all. However, `foo` won't work when called from any crate.

If you have macros, you are encouraged to make sure you have tests that fully cover the macro's syntax. You may also want to test the macros by importing and using them in crates from multiple editions, just to ensure it works correctly everywhere. If you run into issues, you'll need to read through the chapters of this guide to understand how the code can be changed to work across all editions.

## Macro hygiene

Macros use a system called "edition hygiene" where the tokens within a macro are marked with which edition they come from. This allows external macros to be called from crates of varying editions without needing to worry about which edition it is called from.

Let's take a closer look at the example above that defines a `macro_rules` macro using `dyn` as an identifier. If that macro was defined in a crate using the 2015 edition, then that macro works fine, even if it were called from a 2018 crate where `dyn` is a keyword and that would normally be a syntax error. The `let dyn = 1;` tokens are marked as being from 2015, and the compiler will remember that wherever that code gets expanded. The parser looks at the edition of the tokens to know how to interpret it.

The problem arises when changing the edition to 2018 in the crate where it is defined. Now, those tokens are tagged with the 2018 edition, and those will fail to parse. However, since we never called the macro from our crate, `cargo fix --edition` never had a chance to inspect the macro and fix it.

## Documentation tests

At this time, `cargo fix` is not able to update [documentation tests](#). After updating the edition in `Cargo.toml`, you should run `cargo test` to ensure everything still passes. If your documentation tests use syntax that is not supported in the new edition, you will need to update them manually.

In rare cases, you can manually set the edition for each test. For example, you can use the

`edition2018` annotation on the triple backticks to tell `rustdoc` which edition to use.

## Generated code

Another area where the automated fixes cannot apply is if you have a build script which generates Rust code at compile time (see [Code generation](#) for an example). In this situation, if you end up with code that doesn't work in the next edition, you will need to manually change the build script to generate code that is compatible.

## Migrating non-Cargo projects

If your project is not using Cargo as a build system, it may still be possible to make use of the automated lints to assist migrating to the next edition. You can enable the migration lints as described above by enabling the appropriate [lint group](#). For example, you can use the `#![warn(rust_2021_compatibility)]` attribute or the `-Wrust-2021-compatibility` or `--force-warns=rust-2021-compatibility` CLI flag.

The next step is to apply those lints to your code. There are several options here:

- Manually read the warnings and apply the suggestions recommended by the compiler.
- Use an editor or IDE that supports automatically applying suggestions. For example, [Visual Studio Code](#) with the [Rust Analyzer extension](#) has the ability to use the "Quick Fix" links to automatically apply suggestions. Many other editors and IDEs have similar functionality.
- Write a migration tool using the `rustfix` library. This is the library that Cargo uses internally to take the [JSON messages](#) from the compiler and modify the source code. Check the [examples directory](#) for examples of how to use the library.

## Writing idiomatic code in a new edition

Editions are not only about new features and removing old ones. In any programming language, idioms change over time, and Rust is no exception. While old code will continue to compile, it might be written with different idioms today.

For example, in Rust 2015, external crates must be listed with `extern crate` like this:

```
// src/lib.rs
extern crate rand;
```

In Rust 2018, it is [no longer necessary](#) to include these items.

`cargo fix` has the `--edition-idioms` option to automatically transition some of these idioms to the new syntax.

**Warning:** The current *"idiom lints"* are known to have some problems. They may make incorrect suggestions which may fail to compile. The current lints are:

- Edition 2018:
  - `unused-extern-crates`
  - `explicit-outlives-requirements`
- Edition 2021 does not have any idiom lints.

The following instructions are recommended only for the intrepid who are willing to work through a few compiler/Cargo bugs! If you run into problems, you can try the `--broken-code` option [described above](#) to make as much progress as possible, and then resolve the remaining issues manually.

With that out of the way, we can instruct Cargo to fix our code snippet with:

```
cargo fix --edition-idioms
```

Afterwards, the line with `extern crate rand;` in `src/lib.rs` will be removed.

We're now more idiomatic, and we didn't have to fix our code manually!

# Rust 2015

Rust 2015 has a theme of "stability". It commenced with the release of 1.0, and is the "default edition". The edition system was conceived in late 2017, but Rust 1.0 was released in May of 2015. As such, 2015 is the edition that you get when you don't specify any particular edition, for backwards compatibility reasons.

"Stability" is the theme of Rust 2015 because 1.0 marked a huge change in Rust development. Previous to Rust 1.0, Rust was changing on a daily basis. This made it very difficult to write large software in Rust, and made it difficult to learn. With the release of Rust 1.0 and Rust 2015, we committed to backwards compatibility, ensuring a solid foundation for people to build projects on top of.

Since it's the default edition, there's no way to port your code to Rust 2015; it just *is*. You'll be transitioning *away* from 2015, but never really *to* 2015. As such, there's not much else to say about it!



# Rust 2018

| Info            |                                                                |
|-----------------|----------------------------------------------------------------|
| RFC             | <a href="#">#2052</a> , which also proposed the Edition system |
| Release version | <a href="#">1.31.0</a>                                         |

The edition system was created for the release of Rust 2018. The release of the Rust 2018 edition coincided with a number of other features all coordinated around the theme of *productivity*. The majority of those features were backwards compatible and are now available on all editions; however, some of those changes required the edition mechanism (most notably the [module system changes](#)).

# Path and module system changes

Minimum Rust Version 1.31

## Summary

- Paths in `use` declarations now work the same as other paths.
- Paths starting with `::` must now be followed with an external crate.
- Paths in `pub(in path)` visibility modifiers must now start with `crate`, `self`, or `super`.

## Motivation

The module system is often one of the hardest things for people new to Rust. Everyone has their own things that take time to master, of course, but there's a root cause for why it's so confusing to many: while there are simple and consistent rules defining the module system, their consequences can feel inconsistent, counterintuitive and mysterious.

As such, the 2018 edition of Rust introduces a few new module system features, but they end up *simplifying* the module system, to make it more clear as to what is going on.

Here's a brief summary:

- `extern crate` is no longer needed in 99% of circumstances.
- The `crate` keyword refers to the current crate.
- Paths may start with a crate name, even within submodules.
- Paths starting with `::` must reference an external crate.
- A `foo.rs` and `foo/` subdirectory may coexist; `mod.rs` is no longer needed when placing submodules in a subdirectory.
- Paths in `use` declarations work the same as other paths.

These may seem like arbitrary new rules when put this way, but the mental model is now significantly simplified overall. Read on for more details!

# More details

Let's talk about each new feature in turn.

## No more `extern crate`

This one is quite straightforward: you no longer need to write `extern crate` to import a crate into your project. Before:

```
// Rust 2015

extern crate futures;

mod submodule {
    use futures::Future;
}
```

After:

```
// Rust 2018

mod submodule {
    use futures::Future;
}
```

Now, to add a new crate to your project, you can add it to your `Cargo.toml`, and then there is no step two. If you're not using Cargo, you already had to pass `--extern` flags to give `rustc` the location of external crates, so you'd just keep doing what you were doing there as well.

## An exception

There's one exception to this rule, and that's the "sysroot" crates. These are the crates distributed with Rust itself.

Usually these are only needed in very specialized situations. Starting in 1.41, `rustc` accepts the `--extern=CRATE_NAME` flag which automatically adds the given crate name in a way similar to `extern crate`. Build tools may use this to inject sysroot crates into the crate's prelude. Cargo does not have a general way to express this, though it uses it for `proc_macro` crates.

Some examples of needing to explicitly import sysroot crates are:

- `std`: Usually this is not necessary, because `std` is automatically imported unless the crate is marked with `#![no_std]`.
- `core`: Usually this is not necessary, because `core` is automatically imported, unless the crate is marked with `#![no_core]`. For example, some of the internal crates used by the standard library itself need this.
- `proc_macro`: This is automatically imported by Cargo if it is a proc-macro crate starting in 1.42. `extern crate proc_macro;` would be needed if you want to support older releases, or if using another build tool that does not pass the appropriate `--extern` flags to `rustc`.
- `alloc`: Items in the `alloc` crate are usually accessed via re-exports in the `std` crate. If you are working with a `no_std` crate that supports allocation, then you may need to explicitly import `alloc`.
- `test`: This is only available on the [nightly channel](#), and is usually only used for the unstable benchmark support.

## Macros

One other use for `extern crate` was to import macros; that's no longer needed. Macros may be imported with `use` like any other item. For example, the following use of `extern crate`:

```
#[macro_use]
extern crate bar;

fn main() {
    baz!();
}
```

Can be changed to something like the following:

```
use bar::baz;

fn main() {
    baz!();
}
```

## Renaming crates

If you've been using `as` to rename your crate like this:

```
extern crate futures as f;  
  
use f::Future;
```

then removing the `extern crate` line on its own won't work. You'll need to do this:

```
use futures as f;  
  
use self::f::Future;
```

This change will need to happen in any module that uses `f`.

## The `crate` keyword refers to the current crate

In `use` declarations and in other code, you can refer to the root of the current crate with the `crate::` prefix. For instance, `crate::foo::bar` will always refer to the name `bar` inside the module `foo`, from anywhere else in the same crate.

The prefix `::` previously referred to either the crate root or an external crate; it now unambiguously refers to an external crate. For instance, `::foo::bar` always refers to the name `bar` inside the external crate `foo`.

## Extern crate paths

Previously, using an external crate in a module without a `use` import required a leading `::` on the path.

```
// Rust 2015

extern crate chrono;

fn foo() {
    // this works in the crate root
    let x = chrono::Utc::now();
}

mod submodule {
    fn function() {
        // but in a submodule it requires a leading :: if not imported with
        `use`
        let x = ::chrono::Utc::now();
    }
}
```

Now, extern crate names are in scope in the entire crate, including submodules.

```
// Rust 2018

fn foo() {
    // this works in the crate root
    let x = chrono::Utc::now();
}

mod submodule {
    fn function() {
        // crates may be referenced directly, even in submodules
        let x = chrono::Utc::now();
    }
}
```

If you have a local module or item with the same name as an external crate, a path beginning with that name will be taken to refer to the local module or item. To explicitly refer to the external crate, use the `::name` form.

## No more `mod.rs`

In Rust 2015, if you have a submodule:

```
// This `mod` declaration looks for the `foo` module in
// `foo.rs` or `foo/mod.rs`.
mod foo;
```

It can live in `foo.rs` or `foo/mod.rs`. If it has submodules of its own, it *must* be `foo/mod.rs`. So a `bar` submodule of `foo` would live at `foo/bar.rs`.

In Rust 2018 the restriction that a module with submodules must be named `mod.rs` is lifted. `foo.rs` can just be `foo.rs`, and the submodule is still `foo/bar.rs`. This eliminates the special name, and if you have a bunch of files open in your editor, you can clearly see their names, instead of having a bunch of tabs named `mod.rs`.

| Rust 2015                                                    | Rust 2018                                                |
|--------------------------------------------------------------|----------------------------------------------------------|
| <pre>├── lib.rs └── foo/     ├── mod.rs     └── bar.rs</pre> | <pre>├── lib.rs ├── foo.rs └── foo/     └── bar.rs</pre> |

## use paths

Minimum Rust Version 1.32

Rust 2018 simplifies and unifies path handling compared to Rust 2015. In Rust 2015, paths work differently in `use` declarations than they do elsewhere. In particular, paths in `use` declarations would always start from the crate root, while paths in other code implicitly started from the current scope. Those differences didn't have any effect in the top-level module, which meant that everything would seem straightforward until working on a project large enough to have submodules.

In Rust 2018, paths in `use` declarations and in other code work the same way, both in the top-level module and in any submodule. You can use a relative path from the current scope, a path starting from an external crate name, or a path starting with `::`, `crate`, `super`, or `self`.

Code that looked like this:

```
// Rust 2015

extern crate futures;

use futures::Future;

mod foo {
    pub struct Bar;
}

use foo::Bar;

fn my_poll() -> futures::Poll { ... }

enum SomeEnum {
    V1(usize),
    V2(String),
}

fn func() {
    let five = std::sync::Arc::new(5);
    use SomeEnum::*;
    match ... {
        V1(i) => { ... }
        V2(s) => { ... }
    }
}
```

will look exactly the same in Rust 2018, except that you can delete the `extern crate` line:



```
// Rust 2018

use futures::Future;

mod foo {
    pub struct Bar;
}

use foo::Bar;

fn my_poll() -> futures::Poll { ... }

enum SomeEnum {
    V1(usize),
    V2(String),
}

fn func() {
    let five = std::sync::Arc::new(5);
    use SomeEnum::*;
    match ... {
        V1(i) => { ... }
        V2(s) => { ... }
    }
}
```

The same code will also work completely unmodified in a submodule:

```
// Rust 2018

mod submodule {
    use futures::Future;

    mod foo {
        pub struct Bar;
    }

    use foo::Bar;

    fn my_poll() -> futures::Poll { ... }

    enum SomeEnum {
        V1(usize),
        V2(String),
    }

    fn func() {
        let five = std::sync::Arc::new(5);
        use SomeEnum::*;
        match ... {
            V1(i) => { ... }
            V2(s) => { ... }
        }
    }
}
```

This makes it easy to move code around in a project, and avoids introducing additional complexity to multi-module projects.

# Anonymous trait function parameters deprecated

Minimum Rust Version 1.31

## Summary

- [Trait function parameters](#) may use any irrefutable pattern when the function has a body.

## Details

In accordance with RFC [#1685](#), parameters in trait method declarations are no longer allowed to be anonymous.

For example, in the 2015 edition, this was allowed:

```
trait Foo {  
    fn foo(&self, u8);  
}
```

In the 2018 edition, all parameters must be given an argument name (even if it's just `_`):

```
trait Foo {  
    fn foo(&self, baz: u8);  
}
```

# New keywords

Minimum Rust Version 1.27

## Summary

- `dyn` is a [strict keyword](#), in 2015 it is a [weak keyword](#).
- `async` and `await` are [strict keywords](#).
- `try` is a [reserved keyword](#).

## Motivation

### `dyn Trait` for trait objects

The `dyn Trait` feature is the new syntax for using trait objects. In short:

- `Box<Trait>` becomes `Box<dyn Trait>`
- `&Trait` and `&mut Trait` become `&dyn Trait` and `&mut dyn Trait`

And so on. In code:

```
trait Trait {}

impl Trait for i32 {}

// old
fn function1() -> Box<Trait> {
}

// new
fn function2() -> Box<dyn Trait> {
}
```

That's it!

### Why?

Using just the trait name for trait objects turned out to be a bad decision. The current syntax is often ambiguous and confusing, even to veterans, and favors a feature that is not more frequently used than its alternatives, is sometimes slower, and often cannot be used at all when its alternatives can.

Furthermore, with `impl Trait` arriving, "`impl Trait` vs `dyn Trait`" is much more symmetric, and therefore a bit nicer, than "`impl Trait` vs `Trait`". `impl Trait` is explained [here](#).

In the new edition, you should therefore prefer `dyn Trait` to just `Trait` where you need a trait object.

## **async and await**

These keywords are reserved to implement the async-await feature of Rust, which was ultimately [released to stable in 1.39.0](#).

## **try keyword**

The `try` keyword is reserved for use in `try` blocks, which have not (as of this writing) been stabilized ([tracking issue](#))

# Method dispatch for raw pointers to inference variables

## Summary

- The `tyvar_behind_raw_pointer` lint is now a hard error.

## Details

See Rust issue [#46906](#) for details.

# Cargo changes

## Summary

- If there is a target definition in a `Cargo.toml` manifest, it no longer automatically disables automatic discovery of other targets.
- Target paths of the form `src/{target_name}.rs` are no longer inferred for targets where the `path` field is not set.
- `cargo install` for the current directory is no longer allowed, you must specify `cargo install --path .` to install the current package.

# Rust 2021

| Info            |                       |
|-----------------|-----------------------|
| RFC             | <a href="#">#3085</a> |
| Release version | 1.56.0                |

The Rust 2021 Edition contains several changes that bring new capabilities and more consistency to the language, and opens up room for expansion in the future. The following chapters dive into the details of each change, and they include guidance on migrating your existing code.



# Additions to the prelude

## Summary

- The `TryInto`, `TryFrom` and `FromIterator` traits are now part of the prelude.
- This might make calls to trait methods ambiguous which could make some code fail to compile.

## Details

The [prelude of the standard library](#) is the module containing everything that is automatically imported in every module. It contains commonly used items such as `Option`, `Vec`, `drop`, and `Clone`.

The Rust compiler prioritizes any manually imported items over those from the prelude, to make sure additions to the prelude will not break any existing code. For example, if you have a crate or module called `example` containing a `pub struct Option;`, then `use example::*` will make `Option` unambiguously refer to the one from `example`; not the one from the standard library.

However, adding a *trait* to the prelude can break existing code in a subtle way. For example, a call to `x.try_into()` which comes from a `MyTryInto` trait might fail to compile if `std`'s `TryInto` is also imported, because the call to `try_into` is now ambiguous and could come from either trait. This is the reason we haven't added `TryInto` to the prelude yet, since there is a lot of code that would break this way.

As a solution, Rust 2021 will use a new prelude. It's identical to the current one, except for three new additions:

- `std::convert::TryInto`
- `std::convert::TryFrom`
- `std::iter::FromIterator`

The tracking issue [can be found here](#).

# Migration

As a part of the 2021 edition a migration lint, `rust_2021_prelude_collisions`, has been added in order to aid in automatic migration of Rust 2018 codebases to Rust 2021.

In order to migrate your code to be Rust 2021 Edition compatible, run:

```
cargo fix --edition
```

The lint detects cases where functions or methods are called that have the same name as the methods defined in one of the new prelude traits. In some cases, it may rewrite your calls in various ways to ensure that you continue to call the same function you did before.

If you'd like to migrate your code manually or better understand what `cargo fix` is doing, below we've outlined the situations where a migration is needed along with a counter example of when it's not needed.

## Migration needed

### Conflicting trait methods

When two traits that are in scope have the same method name, it is ambiguous which trait method should be used. For example:

```
trait MyTrait<A> {  
    // This name is the same as the `from_iter` method on the `FromIterator`  
    // trait from `std`.  
    fn from_iter(x: Option<A>);  
}  
  
impl<T> MyTrait<()> for Vec<T> {  
    fn from_iter(_: Option<()>) {}  
}  
  
fn main() {  
    // Vec<T> implements both `std::iter::FromIterator` and `MyTrait`  
    // If both traits are in scope (as would be the case in Rust 2021),  
    // then it becomes ambiguous which `from_iter` method to call  
    <Vec<i32>>::from_iter(None);  
}
```

We can fix this by using fully qualified syntax:

```
fn main() {
    // Now it is clear which trait method we're referring to
    <Vec<i32> as MyTrait<()>>::from_iter(None);
}
```

## Inherent methods on `dyn Trait` objects

Some users invoke methods on a `dyn Trait` value where the method name overlaps with a new prelude trait:

```
mod submodule {
    pub trait MyTrait {
        // This has the same name as `TryInto::try_into`
        fn try_into(&self) -> Result<u32, ()>;
    }
}

// `MyTrait` isn't in scope here and can only be referred to through the path
// `submodule::MyTrait`
fn bar(f: Box<dyn submodule::MyTrait>) {
    // If `std::convert::TryInto` is in scope (as would be the case in Rust
    // 2021),
    // then it becomes ambiguous which `try_into` method to call
    f.try_into();
}
```

Unlike with static dispatch methods, calling a trait method on a trait object does not require that the trait be in scope. The code above works as long as there is no trait in scope with a conflicting method name. When the `TryInto` trait is in scope (which is the case in Rust 2021), this causes an ambiguity. Should the call be to `MyTrait::try_into` or `std::convert::TryInto::try_into`?

In these cases, we can fix this by adding an additional dereferences or otherwise clarify the type of the method receiver. This ensures that the `dyn Trait` method is chosen, versus the methods from the prelude trait. For example, turning `f.try_into()` above into `(&*f).try_into()` ensures that we're calling `try_into` on the `dyn MyTrait` which can only refer to the `MyTrait::try_into` method.

## No migration needed

## Inherent methods

Many types define their own inherent methods with the same name as a trait method. For instance, below the struct `MyStruct` implements `from_iter` which shares the same name with the method from the trait `FromIterator` found in the standard library:

```
use std::iter::IntoIterator;

struct MyStruct {
    data: Vec<u32>
}

impl MyStruct {
    // This has the same name as `std::iter::FromIterator::from_iter`
    fn from_iter(iter: impl IntoIterator<Item = u32>) -> Self {
        Self {
            data: iter.into_iter().collect()
        }
    }
}

impl std::iter::FromIterator<u32> for MyStruct {
    fn from_iter<I: IntoIterator<Item = u32>>(iter: I) -> Self {
        Self {
            data: iter.into_iter().collect()
        }
    }
}
```

Inherent methods always take precedent over trait methods so there's no need for any migration.

## Implementation Reference

The lint needs to take a couple of factors into account when determining whether or not introducing 2021 Edition to a codebase will cause a name resolution collision (thus breaking the code after changing edition). These factors include:

- Is the call a [fully-qualified call](#) or does it use [dot-call method syntax](#)?
  - This will affect how the name is resolved due to auto-reference and auto-dereferencing on method call syntax. Manually dereferencing/referencing will allow specifying priority in the case of dot-call method syntax, while fully-qualified call requires specification of the type and the trait name in the

method path (e.g. `<Type as Trait>::method` )

- Is this an **inherent method** or a **trait method**?
  - Inherent methods that take `self` will take priority over `TryInto::try_into` as inherent methods take priority over trait methods, but inherent methods that take `&self` or `&mut self` won't take priority due to requiring a auto-reference (while `TryInto::try_into` does not, as it takes `self` )
- Is the origin of this method from `core` / `std` ? (As the traits can't have a collision with themselves)
- Does the given type implement the trait it could have a collision against?
- Is the method being called via dynamic dispatch? (i.e. is the `self` type `dyn Trait` )
  - If so, trait imports don't affect resolution, and no migration lint needs to occur

# Default Cargo feature resolver

## Summary

- `edition = "2021"` implies `resolver = "2"` in `Cargo.toml`.

## Details

Since Rust 1.51.0, Cargo has opt-in support for a [new feature resolver](#) which can be activated with `resolver = "2"` in `Cargo.toml`.

Starting in Rust 2021, this will be the default. That is, writing `edition = "2021"` in `Cargo.toml` will imply `resolver = "2"`.

The resolver is a global setting for a [workspace](#), and the setting is ignored in dependencies. The setting is only honored for the top-level package of the workspace. If you are using a [virtual workspace](#), you will still need to explicitly set the `resolver` field in the `[workspace]` definition if you want to opt-in to the new resolver.

The new feature resolver no longer merges all requested features for crates that are depended on in multiple ways. See [the announcement of Rust 1.51](#) for details.

## Migration

There are no automated migration tools for updating for the new resolver. For most projects, there are usually few or no changes as a result of updating.

When updating with `cargo fix --edition`, Cargo will display a report if the new resolver will build dependencies with different features. It may look something like this:

note: Switching to Edition 2021 will enable the use of the version 2 feature resolver in Cargo. This may cause some dependencies to be built with fewer features enabled than previously. More information about the resolver changes may be

found at <https://doc.rust-lang.org/nightly/edition-guide/rust-2021/default-cargo-resolver.html>

When building the following dependencies, the given features will no longer be used:

```
bstr v0.2.16: default, lazy_static, regex-automata, unicode
libz-sys v1.1.3 (as host dependency): libc
```

This lets you know that certain dependencies will no longer be built with the given features.

## Build failures

There may be some circumstances where your project may not build correctly after the change. If a dependency declaration in one package assumes that certain features are enabled in another, and those features are now disabled, it may fail to compile.

For example, let's say we have a dependency like this:

```
# Cargo.toml

[dependencies]
bstr = { version = "0.2.16", default-features = false }
# ...
```

And somewhere in our dependency tree, another package has this:

```
# Another package's Cargo.toml

[build-dependencies]
bstr = "0.2.16"
```

In our package, we've been using the `words_with_breaks` method from `bstr`, which requires `bstr`'s "unicode" feature to be enabled. This has historically worked because Cargo unified the features of `bstr` between the two packages. However, after updating to Rust 2021, the new resolver will build `bstr` twice, once with the default features (as a build dependency), and once with no features (as our normal dependency). Since `bstr` is now being built without the "unicode" feature, the `words_with_breaks` method doesn't exist, and the build will fail with an error that the method is missing.

The solution here is to ensure that the dependency is declared with the features you are actually using. For example:

```
[dependencies]
bstr = { version = "0.2.16", default-features = false, features = ["unicode"]
}
```

In some cases, this may be a problem with a third-party dependency that you don't have direct control over. You can consider submitting a patch to that project to try to declare the correct set of features for the problematic dependency. Alternatively, you can add features to any dependency from within your own `Cargo.toml` file. For example, if the `bstr` example given above was declared in some third-party dependency, you can just copy the correct dependency declaration into your own project. The features will be unified, as long as they match the unification rules of the new resolver. Those are:

- Features enabled on platform-specific dependencies for targets not currently being built are ignored.
- Build-dependencies and proc-macros do not share features with normal dependencies.
- Dev-dependencies do not activate features unless building a target that needs them (like tests or examples).

A real-world example is using `diesel` and `diesel_migrations`. These packages provide database support, and the database is selected using a feature, like this:

```
[dependencies]
diesel = { version = "1.4.7", features = ["postgres"] }
diesel_migrations = "1.4.0"
```

The problem is that `diesel_migrations` has an internal proc-macro which itself depends on `diesel`, and the proc-macro assumes its own copy of `diesel` has the same features enabled as the rest of the dependency graph. After updating to the new resolver, it fails to build because now there are two copies of `diesel`, and the one built for the proc-macro is missing the "postgres" feature.

A solution here is to add `diesel` as a build-dependency with the required features, for example:

```
[build-dependencies]
diesel = { version = "1.4.7", features = ["postgres"] }
```



This causes Cargo to add "postgres" as a feature for host dependencies (proc-macros and build-dependencies). Now, the `diesel_migrations` proc-macro will get the "postgres" feature enabled, and it will build correctly.

The 2.0 release of `diesel` (currently in development) does not have this problem as it has been restructured to not have this dependency requirement.

## Exploring features

The `cargo tree` command has had substantial improvements to help with the migration to the new resolver. `cargo tree` can be used to explore the dependency graph, and to see which features are being enabled, and importantly *why* they are being enabled.

One option is to use the `--duplicates` flag (`-d` for short), which will tell you when a package is being built multiple times. Taking the `bstr` example from earlier, we might see:

```
> cargo tree -d
bstr v0.2.16
└─ foo v0.1.0 (/MyProjects/foo)

bstr v0.2.16
[build-dependencies]
└─ bar v0.1.0
    └─ foo v0.1.0 (/MyProjects/foo)
```

This output tells us that `bstr` is built twice, and shows the chain of dependencies that led to its inclusion in both cases.

You can print which features each package is using with the `-f` flag, like this:

```
cargo tree -f '{p} {f}'
```

This tells Cargo to change the "format" of the output, where it will print both the package and the enabled features.

You can also use the `-e` flag to tell it which "edges" to display. For example, `cargo tree -e features` will show in-between each dependency which features are being added by each dependency. This option becomes more useful with the `-i` flag which can be used to "invert" the tree. This allows you to see how features *flow* into a given dependency. For

example, let's say the dependency graph is large, and we're not quite sure who is depending on `bstr`, the following command will show that:

```
> cargo tree -e features -i bstr
bstr v0.2.16
├── bstr feature "default"
│   └── [build-dependencies]
│       └── bar v0.1.0
│           └── bar feature "default"
│               └── foo v0.1.0 (/MyProjects/foo)
├── bstr feature "lazy_static"
│   └── bstr feature "unicode"
│       └── bstr feature "default" (*)
├── bstr feature "regex-automata"
│   └── bstr feature "unicode" (*)
├── bstr feature "std"
│   └── bstr feature "default" (*)
└── bstr feature "unicode" (*)
```

This snippet of output shows that the project `foo` depends on `bar` with the "default" feature. Then, `bar` depends on `bstr` as a build-dependency with the "default" feature. We can further see that `bstr`'s "default" feature enables "unicode" (among other features).

# IntoIterator for arrays

## Summary

- Arrays implement `IntoIterator` in *all* editions.
- Calls to `IntoIterator::into_iter` are *hidden* in Rust 2015 and Rust 2018 when using method call syntax (i.e., `array.into_iter()`). So, `array.into_iter()` still resolves to `(&array).into_iter()` as it has before.
- `array.into_iter()` changes meaning to be the call to `IntoIterator::into_iter` in Rust 2021.

## Details

Until Rust 1.53, only *references* to arrays implement `IntoIterator`. This means you can iterate over `&[1, 2, 3]` and `&mut [1, 2, 3]`, but not over `[1, 2, 3]` directly.

```
for &e in &[1, 2, 3] {} // Ok :)  
  
for e in [1, 2, 3] {} // Error :(
```

This has been a [long-standing issue](#), but the solution is not as simple as it seems. Just adding the trait implementation would break existing code. `array.into_iter()` already compiles today because that implicitly calls `(&array).into_iter()` due to [how method call syntax works](#). Adding the trait implementation would change the meaning.

Usually this type of breakage (adding a trait implementation) is categorized as 'minor' and acceptable. But in this case there is too much code that would be broken by it.

It has been suggested many times to "only implement `IntoIterator` for arrays in Rust 2021". However, this is simply not possible. You can't have a trait implementation exist in one edition and not in another, since editions can be mixed.

Instead, the trait implementation was added in *all* editions (starting in Rust 1.53.0) but with a small hack to avoid breakage until Rust 2021. In Rust 2015 and 2018 code, the compiler will still resolve `array.into_iter()` to `(&array).into_iter()` like before, as if

the trait implementation does not exist. This *only* applies to the `.into_iter()` method call syntax. It does not affect any other syntax such as `for e in [1, 2, 3]`, `iter.zip([1, 2, 3])` or `IntoIterator::into_iter([1, 2, 3])`. Those will start to work in *all* editions.

While it's a shame that this required a small hack to avoid breakage, this solution keeps the difference between the editions to an absolute minimum.

## Migration

A lint, `array_into_iter`, gets triggered whenever there is some call to `into_iter()` that will change meaning in Rust 2021. The `array_into_iter` lint has already been a warning by default on all editions since the 1.41 release (with several enhancements made in 1.55). If your code is already warning free, then it should already be ready to go for Rust 2021!

You can automatically migrate your code to be Rust 2021 Edition compatible or ensure it is already compatible by running:

```
cargo fix --edition
```

Because the difference between editions is small, the migration to Rust 2021 is fairly straight-forward.

For method calls of `into_iter` on arrays, the elements being implemented will change from references to owned values.

For example:

```
fn main() {  
    let array = [1u8, 2, 3];  
    for x in array.into_iter() {  
        // x is a `&u8` in Rust 2015 and Rust 2018  
        // x is a `u8` in Rust 2021  
    }  
}
```

The most straightforward way to migrate in Rust 2021, is by keeping the exact behavior from previous editions by calling `iter()` which also iterates over owned arrays by reference:

```
fn main() {  
    let array = [1u8, 2, 3];  
    for x in array.iter() { // <- This line changed  
        // x is a `&u8` in all editions  
    }  
}
```

## Optional migration

If you are using fully qualified method syntax (i.e., `IntoIterator::into_iter(array)`) in a previous edition, this can be upgraded to method call syntax (i.e., `array.into_iter()`).

# Disjoint capture in closures

## Summary

- `|| a.x + 1` now captures only `a.x` instead of `a`.
- This can cause things to be dropped at different times or affect whether closures implement traits like `Send` or `Clone`.
  - If possible changes are detected, `cargo fix` will insert statements like `let _ = &a` to force a closure to capture the entire variable.

## Details

[Closures](#) automatically capture anything that you refer to from within their body. For example, `|| a + 1` automatically captures a reference to `a` from the surrounding context.

In Rust 2018 and before, closures capture entire variables, even if the closure only uses one field. For example, `|| a.x + 1` captures a reference to `a` and not just `a.x`. Capturing `a` in its entirety prevents mutation or moves from other fields of `a`, so that code like this does not compile:

```
let a = SomeStruct::new();
drop(a.x); // Move out of one field of the struct
println!("{}", a.y); // Ok: Still use another field of the struct
let c = || println!("{}", a.y); // Error: Tries to capture all of `a`
c();
```

Starting in Rust 2021, closures captures are more precise. Typically they will only capture the fields they use (in some cases, they might capture more than just what they use, see the Rust reference for full details). Therefore, the above example will compile fine in Rust 2021.

Disjoint capture was proposed as part of [RFC 2229](#) and the RFC contains details about the motivation.

# Migration

As a part of the 2021 edition a migration lint, `rust_2021_incompatible_closure_captures`, has been added in order to aid in automatic migration of Rust 2018 codebases to Rust 2021.

In order to migrate your code to be Rust 2021 Edition compatible, run:

```
cargo fix --edition
```

Below is an examination of how to manually migrate code to use closure captures that are compatible with Rust 2021 should the automatic migration fail or you would like to better understand how the migration works.

Changing the variables captured by a closure can cause programs to change behavior or to stop compiling in two cases:

- changes to drop order, or when destructors run ([details](#));
- changes to which traits a closure implements ([details](#)).

Whenever any of the scenarios below are detected, `cargo fix` will insert a "dummy let" into your closure to force it to capture the entire variable:

```
let x = (vec![22], vec![23]);
let c = move || {
    // "Dummy let" that forces `x` to be captured in its entirety
    let _ = &x;

    // Otherwise, only `x.0` would be captured here
    println!("{:?}", x.0);
};
```

This is a conservative analysis: in many cases, these dummy lets can be safely removed and your program will work fine.

## Wild Card Patterns

Closures now only capture data that needs to be read, which means the following closures will not capture `x`:

```
let x = 10;
let c = || {
    let _ = x; // no-op
};

let c = || match x {
    _ => println!("Hello World!")
};
```

The `let _ = x` statement here is a no-op, since the `_` pattern completely ignores the right-hand side, and `x` is a reference to a place in memory (in this case, a variable).

This change by itself (capturing fewer values) doesn't trigger any suggestions, but it may do so in conjunction with the "drop order" change below.

**Subtle:** There are other similar expressions, such as the "dummy lets" `let _ = &x` that we insert, which are not no-ops. This is because the right-hand side (`&x`) is not a reference to a place in memory, but rather an expression that must first be evaluated (and whose result is then discarded).

## Drop Order

When a closure takes ownership of a value from a variable `t`, that value is then dropped when the closure is dropped, and not when the variable `t` goes out of scope:

```
{
    let t = (vec![0], vec![0]);

    {
        let c = || move_value(t); // t is moved here
    } // c is dropped, which drops the tuple `t` as well
} // t goes out of scope here
```

The above code will run the same in both Rust 2018 and Rust 2021. However, in cases where the closure only takes ownership of *part* of a variable, there can be differences:



```

{
    let t = (vec![0], vec![0]);

    {
        let c = || {
            // In Rust 2018, captures all of `t`.
            // In Rust 2021, captures only `t.0`
            move_value(t.0);
        };

        // In Rust 2018, `c` (and `t`) are both dropped when we
        // exit this block.
        //
        // In Rust 2021, `c` and `t.0` are both dropped when we
        // exit this block.
    }

    // In Rust 2018, the value from `t` has been moved and is
    // not dropped.
    //
    // In Rust 2021, the value from `t.0` has been moved, but `t.1`
    // remains, so it will be dropped here.
}

```

In most cases, dropping values at different times just affects when memory is freed and is not important. However, some `Drop` impls (aka, destructors) have side-effects, and changing the drop order in those cases can alter the semantics of your program. In such cases, the compiler will suggest inserting a dummy `let` to force the entire variable to be captured.

## Trait implementations

Closures automatically implement the following traits based on what values they capture:

- `Clone`: if all captured values are `Clone`.
- `Auto traits` like `Send`, `Sync`, and `UnwindSafe`: if all captured values implement the given trait.

In Rust 2021, since different values are being captured, this can affect what traits a closure will implement. The migration lints test each closure to see whether it would have implemented a given trait before and whether it still implements it now; if they find that a trait used to be implemented but no longer is, then "dummy lets" are inserted.

For instance, a common way to allow passing around raw pointers between threads is to

wrap them in a struct and then implement `Send` / `Sync` auto trait for the wrapper. The closure that is passed to `thread::spawn` uses the specific fields within the wrapper but the entire wrapper is captured regardless. Since the wrapper is `Send` / `Sync`, the code is considered safe and therefore compiles successfully.

With disjoint captures, only the specific field mentioned in the closure gets captured, which wasn't originally `Send` / `Sync` defeating the purpose of the wrapper.

```
use std::thread;

struct Ptr(*mut i32);
unsafe impl Send for Ptr {}

let mut x = 5;
let px = Ptr(&mut x as *mut i32);

let c = thread::spawn(move || {
    unsafe {
        *(px.0) += 10;
    }
}); // Closure captured px.0 which is not Send
```

# Panic macro consistency

## Summary

- `panic!(..)` now always uses `format_args!(..)`, just like `println!()`.
- `panic!("{}", ..)` is no longer accepted, without escaping the `{}` as `{{}}`.
- `panic!(x)` is no longer accepted if `x` is not a string literal.
  - Use `std::panic::panic_any(x)` to panic with a non-string payload.
  - Or use `panic!("{}", x)` to use `x`'s `Display` implementation.
- The same applies to `assert!(expr, ..)`.

## Details

The `panic!()` macro is one of Rust's most well known macros. However, it has [some subtle surprises](#) that we can't just change due to backwards compatibility.

```
// Rust 2018
panic!("{}", 1); // Ok, panics with the message "1"
panic!("{}", ..); // Ok, panics with the message "{}"
```

The `panic!()` macro only uses string formatting when it's invoked with more than one argument. When invoked with a single argument, it doesn't even look at that argument.

```
// Rust 2018
let a = "{}";
println!(a); // Error: First argument must be a format string literal
panic!(a); // Ok: The panic macro doesn't care
```

It even accepts non-strings such as `panic!(123)`, which is uncommon and rarely useful since it produces a surprisingly unhelpful message: `panicked at 'Box<Any>'`.

This will especially be a problem once [implicit format arguments](#) are stabilized. That feature will make `println!("hello {name}")` a short-hand for `println!("hello {}", name)`. However, `panic!("hello {name}")` would not work as expected, since `panic!()` doesn't process a single argument as format string.

To avoid that confusing situation, Rust 2021 features a more consistent `panic!()` macro. The new `panic!()` macro will no longer accept arbitrary expressions as the only argument. It will, just like `println!()`, always process the first argument as format string. Since `panic!()` will no longer accept arbitrary payloads, `panic_any()` will be the only way to panic with something other than a formatted string.

```
// Rust 2021
panic!("{}", 1); // Ok, panics with the message "1"
panic!("{}",); // Error, missing argument
panic!(a); // Error, must be a string literal
```

In addition, `core::panic!()` and `std::panic!()` will be identical in Rust 2021. Currently, there are some historical differences between those two, which can be noticeable when switching `#![no_std]` on or off.

## Migration

A lint, `non_fmt_panics`, gets triggered whenever there is some call to `panic` that uses some deprecated behavior that will error in Rust 2021. The `non_fmt_panics` lint has already been a warning by default on all editions since the 1.50 release (with several enhancements made in later releases). If your code is already warning free, then it should already be ready to go for Rust 2021!

You can automatically migrate your code to be Rust 2021 Edition compatible or ensure it is already compatible by running:

```
cargo fix --edition
```

Should you choose or need to manually migrate, you'll need to update all panic invocations to either use the same formatting as `println` or use `std::panic::panic_any` to panic with non-string data.

For example, in the case of `panic!(MyStruct)`, you'll need to convert to using `std::panic::panic_any` (note that this is a function not a macro):  
`std::panic::panic_any(MyStruct)`.

In the case of panic messages that include curly braces but the wrong number of arguments (e.g., `panic!("Some curlyes: {}")`), you can panic with the string literal by

either using the same syntax as `println!` (i.e., `panic!("{}", "Some curlies: {}")`) or by escaping the curly braces (i.e., `panic!("Some curlies: {{{}}}")`).

# Reserved syntax

## Summary

- `any_identifier#`, `any_identifier"..."`, `any_identifier'...'`, and `'any_identifier#` are now reserved syntax, and no longer tokenize.
- This is mostly relevant to macros. E.g. `quote!{ #a#b }` is no longer accepted.
- It doesn't treat keywords specially, so e.g. `match"..." {}` is no longer accepted.
- Insert whitespace between the identifier and the subsequent `#`, `"`, or `'` to avoid errors.
- Edition migrations will help you insert whitespace in such cases.

## Details

To make space for new syntax in the future, we've decided to reserve syntax for prefixed identifiers, literals, and lifetimes: `prefix#identifier`, `prefix"string"`, `prefix'c'`, `prefix#123`, and `'prefix#`, where `prefix` can be any identifier. (Except those prefixes that already have a meaning, such as `b'...'` (byte chars) and `r"..."` (raw strings).)

This provides syntax we can expand into in the future without requiring an edition boundary. We may use this for temporary syntax until the next edition, or for permanent syntax if appropriate.

Without an edition, this would be a breaking change, since macros can currently accept syntax such as `hello"world"`, which they will see as two separate tokens: `hello` and `"world"`. The (automatic) fix is simple though: just insert a space: `hello "world"`. Likewise, `prefix#ident` should become `prefix #ident`. Edition migrations will help with this fix.

Other than turning these into a tokenization error, [the RFC](#) does not attach a meaning to any prefix yet. Assigning meaning to specific prefixes is left to future proposals, which will now—thanks to reserving these prefixes—not be breaking changes.

Some new prefixes you might potentially see in the future (though we haven't committed to any of them yet):

- `k#keyword` to allow writing keywords that don't exist yet in the current edition. For example, while `async` is not a keyword in edition 2015, this prefix would've allowed us to accept `k#async` in edition 2015 without having to wait for edition 2018 to reserve `async` as a keyword.
- `f"""` as a short-hand for a format string. For example, `f"hello {name}"` as a short-hand for the equivalent `format!("{}", name)` invocation.
- `s"""` for `String` literals.

## Migration

As a part of the 2021 edition a migration lint, `rust_2021_prefixes_incompatible_syntax`, has been added in order to aid in automatic migration of Rust 2018 codebases to Rust 2021.

In order to migrate your code to be Rust 2021 Edition compatible, run:

```
cargo fix --edition
```

Should you want or need to manually migrate your code, migration is fairly straightforward.

Let's say you have a macro that is defined like so:

```
macro_rules! my_macro {  
    ($a:tt $b:tt) => {};  
}
```

In Rust 2015 and 2018 it's legal for this macro to be called like so with no space between the first token tree and the second:

```
my_macro!(z"hey");
```

This `z` prefix is no longer allowed in Rust 2021, so in order to call this macro, you must add a space after the prefix like so:

```
my_macro!(z "hey");
```

# Raw lifetimes

## Summary

- `'r#ident_or_keyword` is now allowed as a lifetime, which allows using keywords such as `'r#fn`.

## Details

Raw lifetimes are introduced in the 2021 edition to support the ability to migrate to newer editions that introduce new keywords. This is analogous to [raw identifiers](#) which provide the same functionality for identifiers. For example, the 2024 edition introduced the `gen` keyword. Since lifetimes cannot be keywords, this would cause code that use a lifetime `'gen` to fail to compile. Raw lifetimes allow the migration lint to modify those lifetimes to `'r#gen` which do allow keywords.

In editions prior to 2021, raw lifetimes are parsed as separate tokens. For example `'r#foo` is parsed as three tokens: `'r`, `#`, and `foo`.

## Migration

As a part of the 2021 edition a migration lint, `rust_2021_prefixes_incompatible_syntax`, has been added in order to aid in automatic migration of Rust 2018 codebases to Rust 2021.

In order to migrate your code to be Rust 2021 Edition compatible, run:

```
cargo fix --edition
```

Should you want or need to manually migrate your code, migration is fairly straightforward.

Let's say you have a macro that is defined like so:



```
macro_rules! my_macro {  
    ($a:tt $b:tt $c:tt) => {};  
}
```

In Rust 2015 and 2018 it's legal for this macro to be called like so with no space between the tokens:

```
my_macro!('r#foo);
```

In the 2021 edition, this is now parsed as a single token. In order to call this macro, you must add a space before the identifier like so:

```
my_macro!('r# foo);
```

# Warnings promoted to errors

## Summary

- Code that triggered the `bare_trait_objects` and `ellipsis_inclusive_range_patterns` lints will error in Rust 2021.

## Details

Two existing lints are becoming hard errors in Rust 2021, but these lints will remain warnings in older editions.

### **`bare_trait_objects`:**

The use of the `dyn` keyword to identify `trait objects` will be mandatory in Rust 2021.

For example, the following code which does not include the `dyn` keyword in `&MyTrait` will produce an error instead of just a lint in Rust 2021:

```
pub trait MyTrait {}

pub fn my_function(_trait_object: &MyTrait) { // should be `&dyn MyTrait`
    unimplemented!()
}
```

### **`ellipsis_inclusive_range_patterns`:**

The deprecated `...` syntax for inclusive range patterns (i.e., ranges where the end value is *included* in the range) is no longer accepted in Rust 2021. It has been superseded by `..=`, which is consistent with expressions.

For example, the following code which uses `...` in a pattern will produce an error instead of just a lint in Rust 2021:

```
pub fn less_or_eq_to_100(n: u8) -> bool {  
    matches!(n, 0...100) // should be `0..=100`  
}
```

## Migrations

If your Rust 2015 or 2018 code does not produce any warnings for `bare_trait_objects` or `ellipsis_inclusive_range_patterns` and you've not allowed these lints through the use of `#![allow()]` or some other mechanism, then there's no need to migrate.

To automatically migrate any crate that uses `...` in patterns or does not use `dyn` with trait objects, you can run `cargo fix --edition`.

# Or patterns in macro-rules

## Summary

- How patterns work in `macro_rules` macros changes slightly:
  - `$_:pat` in `macro_rules` now matches usage of `|` too: e.g. `A | B`.
  - The new `$_:pat_param` behaves like `$_:pat` did before; it does not match (top level) `|`.
  - `$_:pat_param` is available in all editions.

## Details

Starting in Rust 1.53.0, `patterns` are extended to support `|` nested anywhere in the pattern. This enables you to write `Some(1 | 2)` instead of `Some(1) | Some(2)`. Since this was simply not allowed before, this is not a breaking change.

However, this change also affects `macro_rules` `macros`. Such macros can accept patterns using the `:pat` fragment specifier. Currently, `:pat` does *not* match top level `|`, since before Rust 1.53, not all patterns (at all nested levels) could contain a `|`. Macros that accept patterns like `A | B`, such as `matches!()` use something like `$(($_:pat))|+`.

Because this would potentially break existing macros, the meaning of `:pat` did not change in Rust 1.53.0 to include `|`. Instead, that change happens in Rust 2021. In the new edition, the `:pat` fragment specifier *will* match `A | B`.

`$_:pat` fragments in Rust 2021 cannot be followed by an explicit `|`. Since there are times that one still wishes to match pattern fragments followed by a `|`, the fragment specified `:pat_param` has been added to retain the older behavior.

It's important to remember that editions are *per crate*, so the only relevant edition is the edition of the crate where the macro is defined. The edition of the crate where the macro is used does not change how the macro works.

# Migration

A lint, `rust_2021_incompatible_or_patterns`, gets triggered whenever there is a use of `$_:pat` which will change meaning in Rust 2021.

You can automatically migrate your code to be Rust 2021 Edition compatible or ensure it is already compatible by running:

```
cargo fix --edition
```

If you have a macro which relies on `$_:pat` not matching the top level use of `|` in patterns, you'll need to change each occurrence of `$_:pat` to `$_:pat_param`.

For example:

```
macro_rules! my_macro {
    ($x:pat | $y:pat) => {
        // TODO: implementation
    }
}

// This macro works in Rust 2018 since ` $x:pat ` does not match against `|`:
my_macro!(1 | 2);

// In Rust 2021 however, the ` $_:pat ` fragment matches `|` and is not allowed
// to be followed by a `|`. To make sure this macro still works in Rust 2021
// change the macro to the following:
macro_rules! my_macro {
    ($x:pat_param | $y:pat) => { // <- this line is different
        // TODO: implementation
    }
}
```

# C-string literals

## Summary

- Literals of the form `c"foo"` or `cr"foo"` represent a string of type `&core::ffi::CStr`.

## Details

Starting with Rust 1.77, C-strings can be written using C-string literal syntax with the `c` or `cr` prefix.

Previously, it was challenging to properly produce a valid string literal that could interoperate with C APIs which terminate with a NUL byte. The `cstr` crate was a popular solution, but that required compiling a proc-macro which was quite expensive. Now, C-strings can be written directly using literal syntax notation, which will generate a value of type `&core::ffi::CStr` which is automatically terminated with a NUL byte.

```

assert_eq!(c"hello", CStr::from_bytes_with_nul(b"hello\0").unwrap());
assert_eq!(
    c"byte escapes \xff work",
    CStr::from_bytes_with_nul(b"byte escapes \xff work\0").unwrap()
);
assert_eq!(
    c"unicode escapes \u{00E6} work",
    CStr::from_bytes_with_nul(b"unicode escapes \xc3\xa6 work\0").unwrap()
);
assert_eq!(
    c"unicode characters αβγ encoded as UTF-8",
    CStr::from_bytes_with_nul(
        b"unicode characters \xce\xb1\xce\xb2\xce\xb3 encoded as UTF-8\0"
    )
    .unwrap()
);
assert_eq!(
    c"strings can continue \
      on multiple lines",
    CStr::from_bytes_with_nul(b"strings can continue on multiple
lines\0").unwrap()
);

```

C-strings do not allow interior NUL bytes (such as with a `\0` escape).

Similar to regular strings, C-strings also support "raw" syntax with the `cr` prefix. These raw C-strings do not process backslash escapes which can make it easier to write strings that contain backslashes. Double-quotes can be included by surrounding the quotes with the `#` character. Multiple `#` characters can be used to avoid ambiguity with internal `"#` sequences.

```

assert_eq!(cr"foo", c"foo");
// Number signs can be used to embed interior double quotes.
assert_eq!(cr#"foo"#, c"\foo\");
// This requires two #.
assert_eq!(cr##"foo"###, c"\foo\"#");
// Escapes are not processed.
assert_eq!(cr"C:\foo", c"C:\\foo");

```

See [The Reference](#) for more details.

## Migration

Migration is only necessary for macros which may have been assuming a sequence of tokens that looks similar to `c"..."` or `cr"..."`, which previous to the 2021 edition would tokenize as two separate tokens, but in 2021 appears as a single token.

As part of the [syntax reservation](#) for the 2021 edition, any macro input which may run into this issue should issue a warning from the

`rust_2021_prefixes_incompatible_syntax` migration lint. See that chapter for more detail.



# Rust 2024

| Info            |                       |
|-----------------|-----------------------|
| RFC             | <a href="#">#3501</a> |
| Release version | 1.85.0                |

# Language

The following chapters detail changes to the language in the 2024 Edition.

# RPIT lifetime capture rules

This chapter describes changes related to the **Lifetime Capture Rules 2024** introduced in [RFC 3498](#), including how to use opaque type *precise capturing* (introduced in [RFC 3617](#)) to migrate your code.

## Summary

- In Rust 2024, *all* in-scope generic parameters, including lifetime parameters, are implicitly captured when the `use<..>` bound is not present.
- Uses of the `Captures` trick (`Captures<..>` bounds) and of the outlives trick (e.g. `'_` bounds) can be replaced by `use<..>` bounds (in all editions) or removed entirely (in Rust 2024).

## Details

### Capturing

*Capturing* a generic parameter in an RPIT (return-position impl Trait) opaque type allows for that parameter to be used in the corresponding hidden type. In Rust 1.82, we added `use<..>` bounds that allow specifying explicitly which generic parameters to capture. Those will be helpful for migrating your code to Rust 2024, and will be helpful in this chapter for explaining how the edition-specific implicit capturing rules work. These `use<..>` bounds look like this:

```
fn capture<'a, T>(x: &'a (), y: T) -> impl Sized + use<'a, T> {
    //
    //
    //
    //
    (x, y)
    //~~~~~
    // The hidden type is: `(&'a (), T)`.
    //
    // This type can use `a` and `T` because they were captured.
}
```

The generic parameters that are captured affect how the opaque type can be used. E.g., this is an error because the lifetime is captured despite the fact that the hidden type does not use the lifetime:

```
fn capture<'a>(_: &'a ()) -> impl Sized + use<'a> {}

fn test<'a>(x: &'a ()) -> impl Sized + 'static {
    capture(x)
    //~^ ERROR lifetime may not live long enough
}
```

Conversely, this is OK:

```
fn capture<'a>(_: &'a ()) -> impl Sized + use<> {}

fn test<'a>(x: &'a ()) -> impl Sized + 'static {
    capture(x) //~ OK
}
```

## Edition-specific rules when no `use<..>` bound is present

If the `use<..>` bound is not present, then the compiler uses edition-specific rules to decide which in-scope generic parameters to capture implicitly.

In all editions, all in-scope type and const generic parameters are captured implicitly when the `use<..>` bound is not present. E.g.:

```
fn f_implicit<T, const C: usize>() -> impl Sized {}
//
//          ~~~~~
//          No `use<..>` bound is present here.
//
// In all editions, the above is equivalent to:
fn f_explicit<T, const C: usize>() -> impl Sized + use<T, C> {}
```

In Rust 2021 and earlier editions, when the `use<..>` bound is not present, generic lifetime parameters are only captured when they appear syntactically within a bound in RPIT opaque types in the signature of bare functions and associated functions and methods within inherent impls. However, starting in Rust 2024, these in-scope generic lifetime parameters are unconditionally captured. E.g.:

```
fn f_implicit(_: &()) -> impl Sized {}
// In Rust 2021 and earlier, the above is equivalent to:
fn f_2021(_: &()) -> impl Sized + use<> {}
// In Rust 2024 and later, it's equivalent to:
fn f_2024(_: &()) -> impl Sized + use<'_> {}
```

This makes the behavior consistent with RPIT opaque types in the signature of associated functions and methods within trait impls, uses of RPIT within trait definitions (RPITIT), and opaque `Future` types created by `async fn`, all of which implicitly capture all in-scope generic lifetime parameters in all editions when the `use<..>` bound is not present.

## Outer generic parameters

Generic parameters from an outer impl are considered to be in scope when deciding what is implicitly captured. E.g.:

```

struct S<T, const C: usize>((T, [(); C]));
impl<T, const C: usize> S<T, C> {
    // ~~~~~
    // These generic parameters are in scope.
    fn f_implicit<U>() -> impl Sized {}
    // ~~~~~
    //      ^ This generic is in scope too.
    //      ^
    //      |
    //      No `use<..>` bound is present here.
    //
    // In all editions, it's equivalent to:
    fn f_explicit<U>() -> impl Sized + use<T, U, C> {}
}

```

## Lifetimes from higher-ranked binders

Similarly, generic lifetime parameters introduced into scope by a higher-ranked `for<..>` binder are considered to be in scope. E.g.:

```

trait Tr<'a> { type Ty; }
impl Tr<'_> for () { type Ty = (); }

fn f_implicit() -> impl for<'a> Tr<'a, Ty = impl Copy> {}
// In Rust 2021 and earlier, the above is equivalent to:
fn f_2021() -> impl for<'a> Tr<'a, Ty = impl Copy + use<>> {}
// In Rust 2024 and later, it's equivalent to:
//fn f_2024() -> impl for<'a> Tr<'a, Ty = impl Copy + use<'a>> {}
//
//      ~~~~~
// However, note that the capturing of higher-ranked lifetimes in
// nested opaque types is not yet supported.

```

## Argument position impl Trait (APIT)

Anonymous (i.e. unnamed) generic parameters created by the use of APIT (argument position impl Trait) are considered to be in scope. E.g.:

```
fn f_implicit(_: impl Sized) -> impl Sized {}
//
//      ~~~~~
//      This is called APIT.
//
// The above is *roughly* equivalent to:
fn f_explicit<_0: Sized>(_: _0) -> impl Sized + use<_0> {}
```

Note that the former is not *exactly* equivalent to the latter because, by naming the generic parameter, turbofish syntax can now be used to provide an argument for it. There is no way to explicitly include an anonymous generic parameter in a `use<..>` bound other than by converting it to a named generic parameter.

## Migration

### Migrating while avoiding overcapturing

The `impl_trait_overcaptures` lint flags RPIT opaque types that will capture additional lifetimes in Rust 2024. This lint is part of the `rust-2024-compatibility` lint group which is automatically applied when running `cargo fix --edition`. In most cases, the lint can automatically insert `use<..>` bounds where needed such that no additional lifetimes are captured in Rust 2024.

To migrate your code to be compatible with Rust 2024, run:

```
cargo fix --edition
```

For example, this will change:

```
fn f<'a>(x: &'a ()) -> impl Sized { *x }
```

...into:

```
fn f<'a>(x: &'a ()) -> impl Sized + use<> { *x }
```

Without this `use<>` bound, in Rust 2024, the opaque type would capture the `'a` lifetime parameter. By adding this bound, the migration lint preserves the existing semantics.

## Migrating cases involving APIT

In some cases, the lint cannot make the change automatically because a generic parameter needs to be given a name so that it can appear within a `use<..>` bound. In these cases, the lint will alert you that a change may need to be made manually. E.g., given:

```
fn f<'a>(x: &'a (), y: impl Sized) -> impl Sized { (*x, y) }
//      ^^              ~~~~~
//              This is a use of APIT.
//
//~^ WARN `impl Sized` will capture more lifetimes than possibly intended in
//   edition 2024
//~| NOTE specifically, this lifetime is in scope but not mentioned in the
//   type's bounds
```

The code cannot be converted automatically because of the use of APIT and the fact that the generic type parameter must be named in the `use<..>` bound. To convert this code to Rust 2024 without capturing the lifetime, you must name that type parameter. E.g.:

```
fn f<'a, T: Sized>(x: &'a (), y: T) -> impl Sized + use<T> { (*x, y) }
//      ~~~~~
// The type parameter has been named here.
```

Note that this changes the API of the function slightly as a type argument can now be explicitly provided for this parameter using turbofish syntax. If this is undesired, you might consider instead whether you can simply continue to omit the `use<..>` bound and allow the lifetime to be captured. This might be particularly desirable if you might in the future want to use that lifetime in the hidden type and would like to save space for that.

## Migrating away from the Captures trick

Prior to the introduction of precise capturing `use<..>` bounds in Rust 1.82, correctly capturing a lifetime in an RPIT opaque type often required using the `Captures` trick. E.g.:



```
#[doc(hidden)]
pub trait Captures<T: ?Sized> {}
impl<T: ?Sized, U: ?Sized> Captures<T> for U {}

fn f<'a, T>(x: &'a (), y: T) -> impl Sized + Captures<(&'a (), T)> {
//      ~~~~~
//      This is called the `Captures` trick.
    (x, y)
}
```

With the `use<..>` bound syntax, the `Captures` trick is no longer needed and can be replaced with the following in all editions:

```
fn f<'a, T>(x: &'a (), y: T) -> impl Sized + use<'a, T> {
    (x, y)
}
```

In Rust 2024, the `use<..>` bound can often be omitted entirely, and the above can be written simply as:

```
fn f<'a, T>(x: &'a (), y: T) -> impl Sized {
    (x, y)
}
```

There is no automatic migration for this, and the `Captures` trick still works in Rust 2024, but you might want to consider migrating code manually away from using this old trick.

## Migrating away from the outlives trick

Prior to the introduction of precise capturing `use<..>` bounds in Rust 1.82, it was common to use the "outlives trick" when a lifetime needed to be used in the hidden type of some opaque. E.g.:

```
fn f<'a, T: 'a>(x: &'a (), y: T) -> impl Sized + 'a {
//      ~~~~
//      ^           This is the outlives trick.
//      |
// This bound is needed only for the trick.
    (x, y)
// ~~~~~
// The hidden type is `(&'a (), T)`.
}
```

This trick was less baroque than the `Captures` trick, but also less correct. As we can see in the example above, even though any lifetime components within `T` are independent from the lifetime `'a`, we're required to add a `T: 'a` bound in order to make the trick work. This created undue and surprising restrictions on callers.

Using precise capturing, you can write the above instead, in all editions, as:

```
fn f<T>(x: &(), y: T) -> impl Sized + use<'_, T> {  
    (x, y)  
}
```

In Rust 2024, the `use<..>` bound can often be omitted entirely, and the above can be written simply as:

```
fn f<T>(x: &(), y: T) -> impl Sized {  
    (x, y)  
}
```

There is no automatic migration for this, and the outlives trick still works in Rust 2024, but you might want to consider migrating code manually away from using this old trick.

# if let temporary scope

## Summary

- In an `if let $pat = $expr { .. } else { .. }` expression, the temporary values generated from evaluating `$expr` will be dropped before the program enters the `else` branch instead of after.

## Details

The 2024 Edition changes the drop scope of [temporary values](#) in the scrutinee<sup>1</sup> of an `if let` expression. This is intended to help reduce the potentially unexpected behavior involved with the temporary living for too long.

Before 2024, the temporaries could be extended beyond the `if let` expression itself. For example:

```
// Before 2024

fn f(value: &RwLock<Option<bool>>) {
    if let Some(x) = *value.read().unwrap() {
        println!("value is {x}");
    } else {
        let mut v = value.write().unwrap();
        if v.is_none() {
            *v = Some(true);
        }
    }
    // <--- Read lock is dropped here in 2021
}
```

In this example, the temporary read lock generated by the call to `value.read()` will not be dropped until after the `if let` expression (that is, after the `else` block). In the case where the `else` block is executed, this causes a deadlock when it attempts to acquire a write lock.

The 2024 Edition shortens the lifetime of the temporaries to the point where the then-

block is completely evaluated or the program control enters the `else` block.

```
// Starting with 2024

fn f(value: &RwLock<Option<bool>>) {
    if let Some(x) = *value.read().unwrap() {
        println!("value is {x}");
    }
    // <--- Read lock is dropped here in 2024
    else {
        let mut s = value.write().unwrap();
        if s.is_none() {
            *s = Some(true);
        }
    }
}
```

See the [temporary scope rules](#) for more information about how temporary scopes are extended. See the [tail expression temporary scope](#) chapter for a similar change made to tail expressions.

<sup>1</sup> The [scrutinee](#) is the expression being matched on in the `if let` expression.

## Migration

It is always safe to rewrite `if let` with a `match`. The temporaries of the `match` scrutinee are extended past the end of the `match` expression (typically to the end of the statement), which is the same as the 2021 behavior of `if let`.

The `if_let_rescope` lint suggests a fix when a lifetime issue arises due to this change or the lint detects that a temporary value with a custom, non-trivial `Drop` destructor is generated from the scrutinee of the `if let`. For instance, the earlier example may be rewritten into the following when the suggestion from `cargo fix` is accepted:

```
fn f(value: &RwLock<Option<bool>>) {  
    match *value.read().unwrap() {  
        Some(x) => {  
            println!("value is {x}");  
        }  
        _ => {  
            let mut s = value.write().unwrap();  
            if s.is_none() {  
                *s = Some(true);  
            }  
        }  
    }  
    // <--- Read lock is dropped here in both 2021 and 2024  
}
```

In this particular example, that's probably not what you want due to the aforementioned deadlock! However, some scenarios may be assuming that the temporaries are held past the `else` clause, in which case you may want to retain the old behavior.

The `if_let_rescope` lint is part of the `rust-2024-compatibility` lint group which is included in the automatic edition migration. In order to migrate your code to be Rust 2024 Edition compatible, run:

```
cargo fix --edition
```

After the migration, it is recommended that you review all of the changes of `if let` to `match` and decide what is the behavior that you need with respect to when temporaries are dropped. If you determine that the change is unnecessary, then you can revert the change back to `if let`.

If you want to manually inspect these warnings without performing the edition migration, you can enable the lint with:

```
// Add this to the root of your crate to do a manual migration.  
#![warn(if_let_rescope)]
```

# Tail expression temporary scope

## Summary

- Temporary values generated in evaluation of the tail expression of a [function](#) or closure body, or a [block](#) may now be dropped before local variables, and are sometimes not extended to the next larger temporary scope.

## Details

The 2024 Edition changes the drop order of [temporary values](#) in tail expressions. It often comes as a surprise that, before the 2024 Edition, temporary values in tail expressions can live longer than the block itself, and are dropped later than the local variable bindings, as in the following example:

```
// Before 2024
fn f() -> usize {
    let c = RefCell::new("..");
    c.borrow().len() // error[E0597]: `c` does not live long enough
}
```

This yields the following error with the 2021 Edition:

```

error[E0597]: `c` does not live long enough
--> src/lib.rs:4:5
3 |     let c = RefCell::new("..");
  |         - binding `c` declared here
4 |     c.borrow().len() // error[E0597]: `c` does not live long enough
  |     ^-----
  |     |
  |     borrowed value does not live long enough
  |     a temporary with access to the borrow is created here ...
5 | }
  | -
  | |
  | | `c` dropped here while still borrowed
  | ... and the borrow might be used here, when that temporary is dropped and
  | runs the destructor for type `Ref<'_, &str>`
  |
  | = note: the temporary is part of an expression at the end of a block;
  |         consider forcing this temporary to be dropped sooner, before the
  |         block's local variables are dropped
  | help: for example, you could save the expression's value in a new local
  |       variable `x` and then make `x` be the expression at the end of the block
4 |     let x = c.borrow().len(); x // error[E0597]: `c` does not live long
  |     enough
  |     ++++++          +++

```

For more information about this error, try `rustc --explain E0597`.

In 2021 the local variable `c` is dropped before the temporary created by `c.borrow()`. The 2024 Edition changes this so that the temporary value `c.borrow()` is dropped first, followed by dropping the local variable `c`, allowing the code to compile as expected.

## Temporary scope may be narrowed

When a temporary is created in order to evaluate an expression, the temporary is dropped based on the [temporary scope rules](#). Those rules define how long the temporary will be kept alive. Before 2024, temporaries from tail expressions of a block would be extended outside of the block to the next temporary scope boundary. In many cases this would be the end of a statement or function body. In 2024, the temporaries of the tail expression may now be dropped immediately at the end of the block (before any local variables in the block).

This narrowing of the temporary scope may cause programs to fail to compile in 2024.

For example:

```
// This example works in 2021, but fails to compile in 2024.  
fn main() {  
    let x = { &String::from("1234") }.len();  
}
```

In this example, in 2021, the temporary `String` is extended outside of the block, past the call to `len()`, and is dropped at the end of the statement. In 2024, it is dropped immediately at the end of the block, causing a compile error about the temporary being dropped while borrowed.

The solution for these kinds of situations is to lift the block expression out to a local variable so that the temporary lives long enough:

```
fn main() {  
    let s = { &String::from("1234") };  
    let x = s.len();  
}
```

This particular example takes advantage of [temporary lifetime extension](#). Temporary lifetime extension is a set of specific rules which allow temporaries to live longer than they normally would. Because the `String` temporary is behind a reference, the `String` temporary is extended long enough for the next statement to call `len()` on it.

See the [if let temporary scope](#) chapter for a similar change made to temporary scopes of `if let` expressions.

## Migration

Unfortunately, there are no semantics-preserving rewrites to shorten the lifetime for temporary values in tail expressions<sup>1</sup>. The `tail_expr_drop_order` lint detects if a temporary value with a custom, non-trivial `Drop` destructor is generated in a tail expression. Warnings from this lint will appear when running `cargo fix --edition`, but will otherwise not automatically make any changes. It is recommended to manually inspect the warnings and determine whether or not you need to make any adjustments.

If you want to manually inspect these warnings without performing the edition migration, you can enable the lint with:



```
// Add this to the root of your crate to do a manual migration.  
#![warn(tail_expr_drop_order)]
```

<sup>1</sup> Details are documented at [RFC 3606](#)

# Match ergonomics reservations

## Summary

- Writing `mut`, `ref`, or `ref mut` on a binding is only allowed within a pattern when the pattern leading up to that binding is fully explicit (i.e. when it does not use match ergonomics).
  - Put differently, when the default binding mode is not `move`, writing `mut`, `ref`, or `ref mut` on a binding is an error.
- Reference patterns (`&` or `&mut`) are only allowed within the fully-explicit prefix of a pattern.
  - Put differently, reference patterns can only match against references in the scrutinee when the default binding mode is `move`.

## Details

### Background

Within `match`, `let`, and other constructs, we match a *pattern* against a *scrutinee*. E.g.:

```
let &[&mut [ref x]] = &[&mut [()]]; // x: &()
// ~~~~~ ~~~~~
//      Pattern      Scrutinee
```

Such a pattern is called fully explicit because it does not elide (i.e. "skip" or "pass") any references within the scrutinee. By contrast, this otherwise-equivalent pattern is not fully explicit:

```
let [x] = &[&mut [()]]; // x: &()
```

Patterns such as this are said to be using match ergonomics, originally introduced in [RFC 2005](#).

Under match ergonomics, as we incrementally match a pattern against a scrutinee, we

keep track of the default binding mode. This mode can be one of `move`, `ref mut`, or `ref`, and it starts as `move`. When we reach a binding, unless an explicit binding mode is provided, the default binding mode is used to decide the binding's type.

For example, here we provide an explicit binding mode, causing `x` to be bound by reference:

```
let ref x = (); // &()
```

By contrast:

```
let [x] = &[()]; // &()
```

Here, in the pattern, we pass the outer shared reference in the scrutinee. This causes the default binding mode to switch from `move` to `ref`. Since there is no explicit binding mode specified, the `ref` binding mode is used when binding `x`.

## `mut` restriction

In Rust 2021 and earlier editions, we allow this oddity:

```
let [x, mut y] = &[(), ()]; // x: &(), mut y: ()
```

Here, because we pass the shared reference in the pattern, the default binding mode switches to `ref`. But then, in these editions, writing `mut` on the binding resets the default binding mode to `move`.

This can be surprising as it's not intuitive that mutability should affect the type.

To leave space to fix this, in Rust 2024 it's an error to write `mut` on a binding when the default binding mode is not `move`. That is, `mut` can only be written on a binding when the pattern (leading up to that binding) is fully explicit.

In Rust 2024, we can write the above example as:

```
let &[ref x, mut y] = &[(), ()]; // x: &(), mut y: ()
```

## `ref / ref mut` restriction

In Rust 2021 and earlier editions, we allow:

```
let [ref x] = &[()]; // x: &()
```

Here, the `ref` explicit binding mode is redundant, as by passing the shared reference (i.e. not mentioning it in the pattern), the binding mode switches to `ref`.

To leave space for other language possibilities, we are disallowing explicit binding modes where they are redundant in Rust 2024. We can rewrite the above example as simply:

```
let [x] = &[()]; // x: &()
```

## Reference patterns restriction

In Rust 2021 and earlier editions, we allow this oddity:

```
let [&x, y] = &[&(), &()]; // x: (), y: &&()
```

Here, the `&` in the pattern both matches against the reference on `&()` and resets the default binding mode to `move`. This can be surprising because the single `&` in the pattern causes a larger than expected change in the type by removing both layers of references.

To leave space to fix this, in Rust 2024 it's an error to write `&` or `&mut` in the pattern when the default binding mode is not `move`. That is, `&` or `&mut` can only be written when the pattern (leading up to that point) is fully explicit.

In Rust 2024, we can write the above example as:

```
let [&x, ref y] = &[&(), &()];
```

## Migration

The `rust_2024_incompatible_pat` lint flags patterns that are not allowed in Rust 2024. This lint is part of the `rust-2024-compatibility` lint group which is automatically applied when running `cargo fix --edition`. This lint will automatically convert affected patterns to fully explicit patterns that work correctly in Rust 2024 and in all prior editions.

To migrate your code to be compatible with Rust 2024, run:

```
cargo fix --edition
```

For example, this will convert this...

```
let [x, mut y] = &[(), ()];  
let [ref x] = &[()];  
let [&x, y] = &[&(), &()];
```

...into this:

```
let &[ref x, mut y] = &[(), ()];  
let &[ref x] = &[()];  
let &[&x, ref y] = &[&(), &()];
```

Alternatively, you can manually enable the lint to find patterns that will need to be migrated:

```
// Add this to the root of your crate to do a manual migration.  
#![warn(rust_2024_incompatible_pat)]
```

# Unsafe extern blocks

## Summary

- `extern` blocks must now be marked with the `unsafe` keyword.

## Details

Rust 1.82 added the ability in all editions to mark `extern` blocks with the `unsafe` keyword.<sup>1</sup> Adding the `unsafe` keyword helps to emphasize that it is the responsibility of the author of the `extern` block to ensure that the signatures are correct. If the signatures are not correct, then it may result in undefined behavior.

The syntax for an unsafe `extern` block looks like this:

```
unsafe extern "C" {  
    // sqrt (from libm) may be called with any `f64`  
    pub safe fn sqrt(x: f64) -> f64;  
  
    // strlen (from libc) requires a valid pointer,  
    // so we mark it as being an unsafe fn  
    pub unsafe fn strlen(p: *const std::ffi::c_char) -> usize;  
  
    // this function doesn't say safe or unsafe, so it defaults to unsafe  
    pub fn free(p: *mut core::ffi::c_void);  
  
    pub safe static IMPORTANT_BYTES: [u8; 256];  
}
```

In addition to being able to mark an `extern` block as `unsafe`, you can also specify if individual items in the `extern` block are `safe` or `unsafe`. Items marked as `safe` can be used without an `unsafe` block.

Starting with the 2024 Edition, it is now required to include the `unsafe` keyword on an `extern` block. This is intended to make it very clear that there are safety requirements that must be upheld by the extern definitions.

<sup>1</sup> See [RFC 3484](#) for the original proposal.

## Migration

The `missing_unsafe_on_extern` lint can update `extern` blocks to add the `unsafe` keyword. The lint is part of the `rust-2024-compatibility` lint group which is included in the automatic edition migration. In order to migrate your code to be Rust 2024 Edition compatible, run:

```
cargo fix --edition
```

Just beware that this automatic migration will not be able to verify that the signatures in the `extern` block are correct. It is still your responsibility to manually review their definition.

Alternatively, you can manually enable the lint to find places where there are `unsafe` blocks that need to be updated.

```
// Add this to the root of your crate to do a manual migration.  
#![warn(missing_unsafe_on_extern)]
```

# Unsafe attributes

## Summary

- The following attributes must now be marked as `unsafe`:
  - `export_name`
  - `link_section`
  - `no_mangle`

## Details

Rust 1.82 added the ability in all editions to mark certain attributes as `unsafe` to indicate that they have soundness requirements that must be upheld.<sup>1</sup> The syntax for an unsafe attribute looks like this:

```
// SAFETY: there is no other global function of this name
#[unsafe(no_mangle)]
pub fn example() {}
```

Marking the attribute with `unsafe` highlights that there are safety requirements that must be upheld that the compiler cannot verify on its own.

Starting with the 2024 Edition, it is now required to mark these attributes as `unsafe`. The following section describes the safety requirements for these attributes.

<sup>1</sup> See [RFC 3325](#) for the original proposal.

## Safety requirements

The `no_mangle`, `export_name`, and `link_section` attributes influence the symbol names and linking behavior of items. Care must be taken to ensure that these attributes are used correctly.

Because the set of symbols across all linked libraries is a global namespace, there can be issues if there is a symbol name collision between libraries. Typically this isn't an issue for



normally defined functions because [symbol mangling](#) helps ensure that the symbol name is unique. However, attributes like `export_name` can upset that assumption of uniqueness.

For example, in previous editions the following crashes on most Unix-like platforms despite containing only safe code:

```
fn main() {  
    println!("Hello, world!");  
}  
  
#[export_name = "malloc"]  
fn foo() -> usize { 1 }
```

In the 2024 Edition, it is now required to mark these attributes as unsafe to emphasize that it is required to ensure that the symbol is defined correctly:

```
// SAFETY: There should only be a single definition of the loop symbol.  
#[unsafe(export_name="loop")]  
fn arduino_loop() {  
    // ...  
}
```

## Migration

The `unsafe_attr_outside_unsafe` lint can update these attributes to use the `unsafe(...)` format. The lint is part of the `rust-2024-compatibility` lint group which is included in the automatic edition migration. In order to migrate your code to be Rust 2024 Edition compatible, run:

```
cargo fix --edition
```

Just beware that this automatic migration will not be able to verify that these attributes are being used correctly. It is still your responsibility to manually review their usage.

Alternatively, you can manually enable the lint to find places where these attributes need to be updated.

```
// Add this to the root of your crate to do a manual migration.  
#![warn(unsafe_attr_outside_unsafe)]
```

# unsafe\_op\_in\_unsafe\_fn warning

## Summary

- The `unsafe_op_in_unsafe_fn` lint now warns by default. This warning detects calls to unsafe operations in unsafe functions without an explicit `unsafe {}` block.

## Details

The `unsafe_op_in_unsafe_fn` lint will fire if there are `unsafe operations` in an unsafe function without an explicit `unsafe {}` block.

```
unsafe fn get_unchecked<T>(x: &[T], i: usize) -> &T {  
    x.get_unchecked(i) // WARNING: requires unsafe block  
}
```

The solution is to wrap any unsafe operations in an `unsafe` block:

```
unsafe fn get_unchecked<T>(x: &[T], i: usize) -> &T {  
    unsafe { x.get_unchecked(i) }  
}
```

This change is intended to help protect against accidental use of unsafe operations in an unsafe function. The `unsafe` function keyword was performing two roles. One was to declare that *calling* the function requires unsafe, and that the caller is responsible to uphold additional safety requirements. The other role was to allow the use of unsafe operations inside of the function. This second role was determined to be too risky without explicit `unsafe` blocks.

More information and motivation may be found in [RFC #2585](#).

## Migration

The `unsafe_op_in_unsafe_fn` lint is part of the `rust-2024-compatibility` lint group. In

order to migrate your code to be Rust 2024 Edition compatible, run:

```
cargo fix --edition
```

Alternatively, you can manually enable the lint to find places where unsafe blocks need to be added, or switch it to `allow` to silence the lint completely.

```
// Add this to the root of your crate to do a manual migration.  
#![warn(unsafe_op_in_unsafe_fn)]
```

# Disallow references to static mut

## Summary

- The `static_mut_refs` lint level is now `deny` by default. This checks for taking a shared or mutable reference to a `static mut`.

## Details

The `static_mut_refs` lint detects taking a reference to a `static mut`. In the 2024 Edition, this lint is now `deny` by default to emphasize that you should avoid making these references.

```
static mut X: i32 = 23;
static mut Y: i32 = 24;

unsafe {
    let y = &X;           // ERROR: shared reference to mutable static
    let ref x = X;         // ERROR: shared reference to mutable static
    let (x, y) = (&X, &Y); // ERROR: shared reference to mutable static
}
```

Merely taking such a reference in violation of Rust's mutability XOR aliasing requirement has always been *instantaneous undefined behavior*, **even if the reference is never read from or written to**. Furthermore, upholding mutability XOR aliasing for a `static mut` requires *reasoning about your code globally*, which can be particularly difficult in the face of reentrancy and/or multithreading.

Note that there are some cases where implicit references are automatically created without a visible `&` operator. For example, these situations will also trigger the lint:

```
static mut NUMS: &[u8; 3] = &[0, 1, 2];

unsafe {
    println!("{NUMS:?}"); // ERROR: shared reference to mutable static
    let n = NUMS.len();   // ERROR: shared reference to mutable static
}
```

## Alternatives

Wherever possible, it is **strongly recommended** to use instead an *immutable* `static` of a type that provides *interior mutability* behind some *locally-reasoned abstraction* (which greatly reduces the complexity of ensuring that Rust's mutability XOR aliasing requirement is upheld).

In situations where no locally-reasoned abstraction is possible and you are therefore compelled still to reason globally about accesses to your `static` variable, you must now use raw pointers such as can be obtained via the `&raw const` or `&raw mut` operators. By first obtaining a raw pointer rather than directly taking a reference, (the safety requirements of) accesses through that pointer will be more familiar to `unsafe` developers and can be deferred until/limited to smaller regions of code.

Note that the following examples are just illustrations and are not intended as full-fledged implementations. Do not copy these as-is. There are details for your specific situation that may require alterations to fit your needs. These are intended to help you see different ways to approach your problem.

It is recommended to read the documentation for the specific types in the standard library, the reference on [undefined behavior](#), the [Rustonomicon](#), and if you are having questions to reach out on one of the Rust forums such as the [Users Forum](#).

## Don't use globals

This is probably something you already know, but if possible it is best to avoid mutable global state. Of course this can be a little more awkward or difficult at times, particularly if you need to pass a mutable reference around between many functions.

## Atomics

The [atomic types](#) provide integers, pointers, and booleans that can be used in a `static` (without `mut`).

```
// Chnage from this:
// static mut COUNTER: u64 = 0;
// to this:
static COUNTER: AtomicU64 = AtomicU64::new(0);

fn main() {
    // Be sure to analyze your use case to determine the correct Ordering to
    // use.
    COUNTER.fetch_add(1, Ordering::Relaxed);
}
```

## Mutex or RwLock

When your type is more complex than an atomic, consider using a `Mutex` or `RwLock` to ensure proper access to the global value.

```
// Change from this:
// static mut QUEUE: VecDeque<String> = VecDeque::new();
// to this:
static QUEUE: Mutex<VecDeque<String>> = Mutex::new(VecDeque::new());

fn main() {
    QUEUE.lock().unwrap().push_back(String::from("abc"));
    let first = QUEUE.lock().unwrap().pop_front();
}
```

## OnceLock or LazyLock

If you are using a `static mut` because you need to do some one-time initialization that can't be `const`, you can instead reach for `OnceLock` or `LazyLock` instead.

```
// Instead of some temporary or uninitialized type like:
//     static mut STATE: Option<GlobalState> = None;
// use this instead:
static STATE: LazyLock<GlobalState> = LazyLock::new(|| {
    GlobalState::new()
});

fn main() {
    STATE.example();
}
```

`OnceLock` is similar to `LazyLock`, but can be used if you need to pass information into the constructor, which can work well with single initialization points (like `main`), or if the inputs are available wherever you access the global.

```
static STATE: OnceLock<GlobalState> = OnceLock::new();

fn main() {
    let args = parse_arguments();
    let state = GlobalState::new(args.verbose);
    let _ = STATE.set(state);
    // ...
    STATE.get().unwrap().example();
}
```

## `no_std` one-time initialization

This example is similar to `OnceLock` in that it provides one-time initialization of a global, but it does not require `std` which is useful in a `no_std` context. Assuming your target supports atomics, then you can use an atomic to check for the initialization of the global. The pattern might look something like this:



```

const UNINITIALIZED: usize = 0;
const INITIALIZING: usize = 1;
const INITIALIZED: usize = 2;

static STATE_INITIALIZED: AtomicUsize = AtomicUsize::new(UNINITIALIZED);
static mut STATE: GlobalState = GlobalState::default();

fn set_global_state(state: GlobalState) {
    if STATE_INITIALIZED
        .compare_exchange(
            UNINITIALIZED,
            INITIALIZING,
            Ordering::SeqCst,
            Ordering::SeqCst,
        )
        .is_ok()
    {
        // SAFETY: The reads and writes to STATE are guarded with the
        // INITIALIZED guard.
        unsafe {
            STATE = state;
        }
        STATE_INITIALIZED.store(INITIALIZED, Ordering::SeqCst);
    } else {
        panic!("already initialized, or concurrent initialization");
    }
}

fn get_state() -> &'static GlobalState {
    if STATE_INITIALIZED.load(Ordering::Acquire) != INITIALIZED {
        panic!("not initialized");
    } else {
        // SAFETY: Mutable access is not possible after state has been
        // initialized.
        unsafe { &*&raw const STATE }
    }
}

fn main() {
    let args = parse_arguments();
    let state = GlobalState::new(args.verbose);
    set_global_state(state);
    // ...
    let state = get_state();
    state.example();
}

```

This example assumes you can put some default value in the static before it is initialized

(the `const default` constructor in this example). If that is not possible, consider using either `MaybeUninit`, or dynamic trait dispatch (with a dummy type that implements a trait), or some other approach to have a default placeholder.

There are community-provided crates that can provide similar one-time initialization, such as the `static-cell` crate (which supports targets that do not have atomics by using `portable-atomic`).

## Raw pointers

In some cases you can continue to use `static mut`, but avoid creating references. For example, if you just need to pass `raw pointers` into a C library, don't create an intermediate reference. Instead you can use `raw borrow operators`, like in the following example:

```
static mut STATE: GlobalState = GlobalState::new();

unsafe extern "C" {
    fn example_ffi(state: *mut GlobalState);
}

fn main() {
    unsafe {
        // Change from this:
        //     example_ffi(&mut STATE as *mut GlobalState);
        // to this:
        example_ffi(&raw mut STATE);
    }
}
```

Just beware that you still need to uphold the aliasing constraints around mutable pointers. This may require some internal or external synchronization or proofs about how it is used across threads, interrupt handlers, and reentrancy.

## UnsafeCell with Sync

`UnsafeCell` does not impl `Sync`, so it cannot be used in a `static`. You can create your own wrapper around `UnsafeCell` to add a `Sync` impl so that it can be used in a `static` to implement interior mutability. This approach can be useful if you have external locks

or other guarantees that uphold the safety invariants required for mutable pointers.

Note that this is largely the same as the [raw pointers](#) example. The wrapper helps to emphasize how you are using the type, and focus on which safety requirements you should be careful of. But otherwise they are roughly the same.

```
#[repr(transparent)]
pub struct SyncUnsafeCell<T>(UnsafeCell<T>);

unsafe impl<T: Sync> Sync for SyncUnsafeCell<T> {}

static STATE: SyncUnsafeCell<GlobalState> =
    SyncUnsafeCell(UnsafeCell::new(GlobalState::new()));

fn set_value(value: i32) {
    with_interrupts_disabled(|| {
        let state = STATE.0.get();
        unsafe {
            // SAFETY: This value is only ever read in our interrupt handler,
            // and interrupts are disabled, and we only use this in one
            thread.
            (*state).value = value;
        }
    });
}
```

The standard library has a nightly-only (unstable) variant of [UnsafeCell](#) called [SyncUnsafeCell](#). This example above shows a very simplified version of the standard library type, but would be used roughly the same way. It can provide even better isolation, so do check out its implementation for more details.

This example includes a fictional [with\\_interrupts\\_disabled](#) function which is the type of thing you might see in an embedded environment. For example, the [critical-section](#) crate provides a similar kind of functionality that could be used for an embedded environment.

## Safe references

In some cases it may be safe to create a reference of a [static mut](#). The whole point of the [static\\_mut\\_refs](#) lint is that this is very hard to do correctly! However, that's not to say it is *impossible*. If you have a situation where you can guarantee that the aliasing requirements are upheld, such as guaranteeing the static is narrowly scoped (only used

in a small module or function), has some internal or external synchronization, accounts for interrupt handlers and reentrancy, panic safety, drop handlers, etc., then taking a reference may be fine.

There are two approaches you can take for this. You can either allow the `static_mut_refs` lint (preferably as narrowly as you can), or convert raw pointers to a reference, as with `&mut *&raw mut MY_STATIC`.

## Short-lived references

If you must create a reference to a `static mut`, then it is recommended to minimize the scope of how long that reference exists. Avoid squirreling the reference away somewhere, or keeping it alive through a large section of code. Keeping it short-lived helps with auditing, and verifying that exclusive access is maintained for the duration. Using pointers should be your default unit, and only convert the pointer to a reference on demand when absolutely required.

## Migration

There is no automatic migration to fix these references to `static mut`. To avoid undefined behavior you must rewrite your code to use a different approach as recommended in the [Alternatives](#) section.

# Never type fallback change

## Summary

- Never type ( `!` ) to any type ("never-to-any") coercions fall back to never type ( `!` ) rather than to unit type ( `()` ).
- The `never_type_fallback_flowng_into_unsafe` lint is now `deny` by default.

## Details

When the compiler sees a value of type `!` (never) in a [coercion site](#), it implicitly inserts a coercion to allow the type checker to infer any type:

```
// This:
let x: u8 = panic!();

// ...is (essentially) turned by the compiler into:
let x: u8 = absurd(panic!());

// ...where `absurd` is the following function
// (it's sound because `!` always marks unreachable code):
fn absurd<T>(x: !) -> T { x }
```

This can lead to compilation errors if the type cannot be inferred:

```
// This:
{ panic!() };

// ...gets turned into this:
{ absurd(panic!()) }; //~ ERROR can't infer the type of `absurd`
```

To prevent such errors, the compiler remembers where it inserted `absurd` calls, and if it can't infer the type, it uses the fallback type instead:

```
type Fallback = /* An arbitrarily selected type! */ !;
{ absurd::<Fallback>(panic!()) }
```

This is what is known as "never type fallback".

Historically, the fallback type has been `()` (unit). This caused `!` to spontaneously coerce to `()` even when the compiler would not infer `()` without the fallback. That was confusing and has prevented the stabilization of the `!` type.

In the 2024 edition, the fallback type is now `!`. (We plan to make this change across all editions at a later date.) This makes things work more intuitively. Now when you pass `!` and there is no reason to coerce it to something else, it is kept as `!`.

In some cases your code might depend on the fallback type being `()`, so this can cause compilation errors or changes in behavior.

## never\_type\_fallback\_flowinto\_unsafe

The default level of the `never_type_fallback_flowinto_unsafe` lint has been raised from `warn` to `deny` in the 2024 Edition. This lint helps detect a particular interaction with the fallback to `!` and `unsafe` code which may lead to undefined behavior. See the link for a complete description.

## Migration

There is no automatic fix, but there is automatic detection of code that will be broken by the edition change. While still on a previous edition you will see warnings if your code will be broken.

The fix is to specify the type explicitly so that the fallback type is not used. Unfortunately, it might not be trivial to see which type needs to be specified.

One of the most common patterns broken by this change is using `f()?;` where `f` is generic over the `Ok`-part of the return type:

```
fn f<T: Default>() -> Result<T, ()> {  
    Ok(T::default())  
}  
  
f()?;
```

You might think that, in this example, type `T` can't be inferred. However, due to the current desugaring of the `?` operator, it was inferred as `()`, and it will now be inferred as `!`.

To fix the issue you need to specify the `T` type explicitly:

```
f::<()>()?;
// ...or:
() = f()?;
```

Another relatively common case is panicking in a closure:

```
trait Unit {}
impl Unit for () {}

fn run<R: Unit>(f: impl FnOnce() -> R) {
    f();
}

run(|| panic!());
```

Previously `!` from the `panic!` coerced to `()` which implements `Unit`. However now the `!` is kept as `!` so this code fails because `!` doesn't implement `Unit`. To fix this you can specify the return type of the closure:

```
run(|| -> () { panic!() });
```

A similar case to that of `f()?` can be seen when using a `!`-typed expression in one branch and a function with an unconstrained return type in the other:

```
if true {
    Default::default()
} else {
    return
};
```

Previously `()` was inferred as the return type of `Default::default()` because `!` from `return` was spuriously coerced to `()`. Now, `!` will be inferred instead causing this code to not compile because `!` does not implement `Default`.

Again, this can be fixed by specifying the type explicitly:

```
() = if true {  
    Default::default()  
} else {  
    return  
};  
  
// ...or:  
  
if true {  
    <() as Default>::default()  
} else {  
    return  
};
```



# Macro Fragment Specifiers

## Summary

- The `expr` fragment specifier now also supports `const` and `_` expressions.
- The `expr_2021` fragment specifier has been added for backwards compatibility.

## Details

As new syntax is added to Rust, existing `macro_rules` fragment specifiers are sometimes not allowed to match on the new syntax in order to retain backwards compatibility. Supporting the new syntax in the old fragment specifiers is sometimes deferred until the next edition, which provides an opportunity to update them.

Indeed this happened with `const expressions` added in 1.79 and `_ expressions` added in 1.59. In the 2021 Edition and earlier, the `expr` fragment specifier does *not* match those expressions. This is because you may have a scenario like:

```
macro_rules! example {
    ($e:expr) => { println!("first rule"); };
    (const $e:expr) => { println!("second rule"); };
}

fn main() {
    example!(const { 1 + 1 });
}
```

Here, in the 2021 Edition, the macro will match the *second* rule. If earlier editions had changed `expr` to match the newly introduced `const` expressions, then it would match the *first* rule, which would be a breaking change.

In the 2024 Edition, `expr` specifiers now also match `const` and `_` expressions. To support the old behavior, the `expr_2021` fragment specifier has been added which does *not* match the new expressions.

# Migration

The `edition_2024_expr_fragment_specifier` lint will change all uses of the `expr` specifier to `expr_2021` to ensure that the behavior of existing macros does not change. The lint is part of the `rust-2024-compatibility` lint group which is included in the automatic edition migration. In order to migrate your code to be Rust 2024 Edition compatible, run:

```
cargo fix --edition
```

In *most* cases, you will likely want to keep the `expr` specifier instead, in order to support the new expressions. You will need to review your macro to determine if there are other rules that would otherwise match with `const` or `_` and determine if there is a conflict. If you want the new behavior, just revert any changes made by the lint.

Alternatively, you can manually enable the lint to find macros where you may need to update the `expr` specifier.

```
// Add this to the root of your crate to do a manual migration.  
#![warn(edition_2024_expr_fragment_specifier)]
```

# Missing macro fragment specifiers

## Summary

- The `missing_fragment_specifier` lint is now a hard error.

## Details

The `missing_fragment_specifier` lint detects a situation when an **unused** pattern in a `macro_rules!` macro definition has a meta-variable (e.g. `$e`) that is not followed by a fragment specifier (e.g. `:expr`). This is now a hard error in the 2024 Edition.

```
macro_rules! foo {  
    () => {};  
    ($name) => { }; // ERROR: missing fragment specifier  
}  
  
fn main() {  
    foo!();  
}
```

Calling the macro with arguments that would match a rule with a missing specifier (e.g., `foo!($name)`) is a hard error in all editions. However, simply defining a macro with missing fragment specifiers is not, though we did add a lint in Rust 1.17.

We'd like to make this a hard error in all editions, but there would be too much breakage right now. So we're starting by making this a hard error in Rust 2024.<sup>1</sup>

<sup>1</sup> The lint is marked as a "future-incompatible" warning to indicate that it may become a hard error in all editions in a future release. See [#40107](#) for more information.

## Migration

To migrate your code to the 2024 Edition, remove the unused matcher rule from the macro. The `missing_fragment_specifier` lint is on by default in all editions, and should

alert you to macros with this issue.

There is no automatic migration for this change. We expect that this style of macro is extremely rare. The lint has been a future-incompatibility lint since Rust 1.17, a deny-by-default lint since Rust 1.20, and since Rust 1.82, it has warned about dependencies that are using this pattern.

# gen keyword

## Summary

- `gen` is a [reserved keyword](#).

## Details

The `gen` keyword has been reserved as part of [RFC #3513](#) to introduce "gen blocks" in a future release of Rust. `gen` blocks will provide a way to make it easier to write certain kinds of iterators. Reserving the keyword now will make it easier to stabilize `gen` blocks before the next edition.

## Migration

Introducing the `gen` keyword can cause a problem for any identifiers that are already called `gen`. For example, any variable or function name called `gen` would clash with the new keyword. To overcome this, Rust supports the `r#` prefix for a [raw identifier](#), which allows identifiers to overlap with keywords.

The `keyword_idents_2024` lint will automatically modify any identifier named `gen` to be `r#gen` so that code continues to work on both editions. This lint is part of the `rust-2024-compatibility` lint group, which will automatically be applied when running `cargo fix --edition`. To migrate your code to be Rust 2024 Edition compatible, run:

```
cargo fix --edition
```

For example, this will change:

```
fn gen() {  
    println!("generating!");  
}  
  
fn main() {  
    gen();  
}
```

to be:

```
fn r#gen() {  
    println!("generating!");  
}  
  
fn main() {  
    r#gen();  
}
```

Alternatively, you can manually enable the lint to find places where `gen` identifiers need to be modified to `r#gen`:

```
// Add this to the root of your crate to do a manual migration.  
#![warn(keyword_idents_2024)]
```

# Reserved syntax

## Summary

- Unprefixed guarded strings of the form `#"foo"#` are reserved for future use.
- Two or more `#` characters are reserved for future use.

## Details

[RFC 3593](#) reserved syntax in the 2024 Edition for guarded string literals that do not have a prefix to make room for possible future language changes. The 2021 Edition [reserved syntax](#) for guarded strings with a prefix, such as `ident##"foo"##`. The 2024 Edition extends that to also reserve strings without the `ident` prefix.

There are two reserved syntaxes:

- One or more `#` characters immediately followed by a [string literal](#).
- Two or more `#` characters in a row (not separated by whitespace).

This reservation is done across an edition boundary because of interactions with tokenization and macros. For example, consider this macro:

```
macro_rules! demo {
  ( $a:tt ) => { println!("one token") };
  ( $a:tt $b:tt $c:tt ) => { println!("three tokens") };
}

demo!("foo");
demo!(r#"foo"#);
demo!(#"foo"#);
demo!(###)
```

Prior to the 2024 Edition, this produces:

```
one token
one token
three tokens
three tokens
```

Starting in the 2024 Edition, the `#"foo"#` line and the `###` line now generates a compile error because those forms are now reserved.

## Migration

The `rust_2024_guarded_string_incompatible_syntax` lint will identify any tokens that match the reserved syntax, and will suggest a modification to insert spaces where necessary to ensure the tokens continue to be parsed separately.

The lint is part of the `rust-2024-compatibility` lint group which is included in the automatic edition migration. In order to migrate your code to be Rust 2024 Edition compatible, run:

```
cargo fix --edition
```

Alternatively, you can manually enable the lint to find macro calls where you may need to update the tokens:

```
// Add this to the root of your crate to do a manual migration.
#![warn(rust_2024_guarded_string_incompatible_syntax)]
```



# Standard library

The following chapters detail changes to the standard library in the 2024 Edition.

# Changes to the prelude

## Summary

- The `Future` and `IntoFuture` traits are now part of the prelude.
- This might make calls to trait methods ambiguous which could make some code fail to compile.
- `RustcEncodable` and `RustcDecodable` have been removed from the prelude.

## Details

The `prelude of the standard library` is the module containing everything that is automatically imported in every module. It contains commonly used items such as `Option`, `Vec`, `drop`, and `Clone`.

The Rust compiler prioritizes any manually imported items over those from the prelude, to make sure additions to the prelude will not break any existing code. For example, if you have a crate or module called `example` containing a `pub struct Option;`, then `use example::*` will make `Option` unambiguously refer to the one from `example`; not the one from the standard library.

However, adding a *trait* to the prelude can break existing code in a subtle way. For example, a call to `x.poll()` which comes from a `MyPoller` trait might fail to compile if `std`'s `Future` is also imported, because the call to `poll` is now ambiguous and could come from either trait.

As a solution, Rust 2024 will use a new prelude. It's identical to the current one, except for the following changes:

- Added:
  - `std::future::Future`
  - `std::future::IntoFuture`
- Removed:
  - `RustcEncodable`
  - `RustcDecodable`

## RustcEncodable and RustcDecodable removal

`RustcEncodable` and `RustcDecodable` are two undocumented derive macros that have been removed from the prelude. These were deprecated before Rust 1.0, but remained within the standard library prelude. The 2024 Edition has removed these from the prelude since they are not expected to be used.

If in the unlikely case there is a project still using these, it is recommended to switch to a serialization library, such as those found on [crates.io](https://crates.io).

## Migration

### Conflicting trait methods

When two traits that are in scope have the same method name, it is ambiguous which trait method should be used. For example:

```
trait MyPoller {  
    // This name is the same as the `poll` method on the `Future` trait from  
    // `std`.  
    fn poll(&self) {  
        println!("polling");  
    }  
}  
  
impl<T> MyPoller for T {}  
  
fn main() {  
    // Pin<&mut async {}> implements both `std::future::Future` and  
    // `MyPoller`.  
    // If both traits are in scope (as would be the case in Rust 2024),  
    // then it becomes ambiguous which `poll` method to call  
    core::pin::pin!(async {}).poll();  
}
```

We can fix this so that it works on all editions by using fully qualified syntax:

```
fn main() {  
    // Now it is clear which trait method we're referring to  
    <_ as MyPoller>::poll(&core::pin::pin!(async {}));  
}
```

The `rust_2024_prelude_collisions` lint will automatically modify any ambiguous method calls to use fully qualified syntax. This lint is part of the `rust-2024-compatibility` lint group, which will automatically be applied when running `cargo fix --edition`. To migrate your code to be Rust 2024 Edition compatible, run:

```
cargo fix --edition
```

Alternatively, you can manually enable the lint to find places where these qualifications need to be added:

```
// Add this to the root of your crate to do a manual migration.  
#![warn(rust_2024_prelude_collisions)]
```

## RustcEncodable and RustcDecodable

It is strongly recommended that you migrate to a different serialization library if you are still using these. However, these derive macros are still available in the standard library, they are just required to be imported from the older prelude now:

```
#[allow(soft_unstable)]  
use core::prelude::v1::{RustcDecodable, RustcEncodable};
```

There is no automatic migration for this change; you will need to make the update manually.

# Add IntoIterator for Box<[T]>

## Summary

- Boxed slices implement `IntoIterator` in *all* editions.
- Calls to `IntoIterator::into_iter` are *hidden* in editions prior to 2024 when using method call syntax (i.e., `boxed_slice.into_iter()`). So, `boxed_slice.into_iter()` still resolves to `(&(*boxed_slice)).into_iter()` as it has before.
- `boxed_slice.into_iter()` changes meaning to call `IntoIterator::into_iter` in Rust 2024.

## Details

Until Rust 1.80, `IntoIterator` was not implemented for boxed slices. In prior versions, if you called `.into_iter()` on a boxed slice, the method call would automatically dereference from `Box<[T]>` to `&[T]`, and return an iterator that yielded references of `&T`. For example, the following worked in prior versions:

```
// Example of behavior in previous editions.
let my_boxed_slice: Box<[u32]> = vec![1, 2, 3].into_boxed_slice();
// Note: .into_iter() was required in versions older than 1.80
for x in my_boxed_slice.into_iter() {
    // x is of type &u32 in editions prior to 2024
}
```

In Rust 1.80, implementations of `IntoIterator` were added for boxed slices. This allows iterating over elements of the slice by-value instead of by-reference:

```
// NEW as of 1.80, all editions
let my_boxed_slice: Box<[u32]> = vec![1, 2, 3].into_boxed_slice();
for x in my_boxed_slice { // notice no need for calling .into_iter()
    // x is of type u32
}
```

This example is allowed on all editions because previously this was an error since `for` loops do not automatically dereference like the `.into_iter()` method call does.

However, this would normally be a breaking change because existing code that manually called `.into_iter()` on a boxed slice would change from having an iterator over references to an iterator over values. To resolve this problem, method calls of `.into_iter()` on boxed slices have edition-dependent behavior. In editions before 2024, it continues to return an iterator over references, and starting in Edition 2024 it returns an iterator over values.

```
// Example of changed behavior in Edition 2024
let my_boxed_slice: Box<[u32]> = vec![1, 2, 3].into_boxed_slice();
// Example of old code that still manually calls .into_iter()
for x in my_boxed_slice.into_iter() {
    // x is now type u32 in Edition 2024
}
```

## Migration

The `boxed_slice_into_iter` lint will automatically modify any calls to `.into_iter()` on boxed slices to call `.iter()` instead to retain the old behavior of yielding references. This lint is part of the `rust-2024-compatibility` lint group, which will automatically be applied when running `cargo fix --edition`. To migrate your code to be Rust 2024 Edition compatible, run:

```
cargo fix --edition
```

For example, this will change:

```
fn main() {
    let my_boxed_slice: Box<[u32]> = vec![1, 2, 3].into_boxed_slice();
    for x in my_boxed_slice.into_iter() {
        // x is of type &u32
    }
}
```

to be:

```
fn main() {  
    let my_boxed_slice: Box<[u32]> = vec![1, 2, 3].into_boxed_slice();  
    for x in my_boxed_slice.iter() {  
        // x is of type &u32  
    }  
}
```

The `boxed_slice_into_iter` lint is defaulted to warn on all editions, so unless you have manually silenced the lint, you should already see it before you migrate.

# Unsafe functions

## Summary

- The following functions are now marked `unsafe`:
  - `std::env::set_var`
  - `std::env::remove_var`
  - `std::os::unix::process::CommandExt::before_exec`

## Details

Over time it has become evident that certain functions in the standard library should have been marked as `unsafe`. However, adding `unsafe` to a function can be a breaking change since it requires existing code to be placed in an `unsafe` block. To avoid the breaking change, these functions are marked as `unsafe` starting in the 2024 Edition, while not requiring `unsafe` in previous editions.

### `std::env::{set_var, remove_var}`

It can be unsound to call `std::env::set_var` or `std::env::remove_var` in a multi-threaded program due to safety limitations of the way the process environment is handled on some platforms. The standard library originally defined these as safe functions, but it was later determined that was not correct.

It is important to ensure that these functions are not called when any other thread might be running. See the [Safety](#) section of the function documentation for more details.

### `std::os::unix::process::CommandExt::before_exec`

The `std::os::unix::process::CommandExt::before_exec` function is a unix-specific function which provides a way to run a closure before calling `exec`. This function was deprecated in the 1.37 release, and replaced with `pre_exec` which does the same thing, but is marked as `unsafe`.



Even though `before_exec` is deprecated, it is now correctly marked as `unsafe` starting in the 2024 Edition. This should help ensure that any legacy code which has not already migrated to `pre_exec` to require an `unsafe` block.

There are very strict safety requirements for the `before_exec` closure to satisfy. See the [Safety section](#) for more details.

## Migration

To make your code compile in both the 2021 and 2024 editions, you will need to make sure that these functions are called only from within `unsafe` blocks.

⚠ **Caution:** It is important that you manually inspect the calls to these functions and possibly rewrite your code to satisfy the preconditions of those functions. In particular, `set_var` and `remove_var` should not be called if there might be multiple threads running. You may need to elect to use a different mechanism other than environment variables to manage your use case.

The `deprecated_safe_2024` lint will automatically modify any use of these functions to be wrapped in an `unsafe` block so that it can compile on both editions. This lint is part of the `rust-2024-compatibility` lint group, which will automatically be applied when running `cargo fix --edition`. To migrate your code to be Rust 2024 Edition compatible, run:

```
cargo fix --edition
```

For example, this will change:

```
fn main() {  
    std::env::set_var("FOO", "123");  
}
```

to be:

```
fn main() {  
    // TODO: Audit that the environment access only happens in single-  
    threaded code.  
    unsafe { std::env::set_var("FOO", "123") };  
}
```

Just beware that this automatic migration will not be able to verify that these functions are being used correctly. It is still your responsibility to manually review their usage.

Alternatively, you can manually enable the lint to find places these functions are called:

```
// Add this to the root of your crate to do a manual migration.  
#![warn(deprecated_safe_2024)]
```

# Cargo

The following chapters detail changes to Cargo in the 2024 Edition.

# Cargo: Rust-version aware resolver

## Summary

- `edition = "2024"` implies `resolver = "3"` in `Cargo.toml` which enables a Rust-version aware dependency resolver.

## Details

Since Rust 1.84.0, Cargo has opt-in support for compatibility with `package.rust-version` to be considered when selecting dependency versions by setting `resolver.incompatible-rust-version = "fallback"` in `.cargo/config.toml`.

Starting in Rust 2024, this will be the default. That is, writing `edition = "2024"` in `Cargo.toml` will imply `resolver = "3"` which will imply `resolver.incompatible-rust-version = "fallback"`.

The resolver is a global setting for a [workspace](#), and the setting is ignored in dependencies. The setting is only honored for the top-level package of the workspace. If you are using a [virtual workspace](#), you will still need to explicitly set the `resolver` field in the `[workspace]` definition if you want to opt-in to the new resolver.

For more details on how Rust-version aware dependency resolution works, see [the Cargo book](#).

## Migration

There are no automated migration tools for updating for the new resolver.

We recommend projects [verify against the latest dependencies in CI](#) to catch bugs in dependencies as soon as possible.

# Cargo: Table and key name consistency

## Summary

- Several table and key names in `Cargo.toml` have been removed where there were previously two ways to specify the same thing.
  - Removed `[project]` ; use `[package]` instead.
  - Removed `default_features` ; use `default-features` instead.
  - Removed `crate_type` ; use `crate-type` instead.
  - Removed `proc_macro` ; use `proc-macro` instead.
  - Removed `dev_dependencies` ; use `dev-dependencies` instead.
  - Removed `build_dependencies` ; use `build-dependencies` instead.

## Details

Several table and keys names are no longer allowed in the 2024 Edition. There were two ways to specify these tables or keys, and this helps ensure there is only one way to specify them.

Some were due to a change in decisions over time, and some were inadvertent implementation artifacts. In order to avoid confusion, and to enforce a single style for specifying these tables and keys, only one variant is now allowed.

For example:

```
[dev_dependencies]
rand = { version = "0.8.5", default_features = false }
```

Should be changed to:

```
[dev-dependencies]
rand = { version = "0.8.5", default-features = false }
```

Notice that the underscores were changed to dashes for `dev_dependencies` and `default_features` .

# Migration

When using `cargo fix --edition`, Cargo will automatically update your `Cargo.toml` file to use the preferred table and key names.

If you would prefer to update your `Cargo.toml` manually, be sure to go through the list above and make sure only the new forms are used.

# Cargo: Reject unused inherited default-features

## Summary

- `default-features = false` is no longer allowed in an inherited workspace dependency if the workspace dependency specifies `default-features = true` (or does not specify `default-features`).

## Details

[Workspace inheritance](#) allows you to specify dependencies in one place (the workspace), and then to refer to those workspace dependencies from within a package. There was an inadvertent interaction with how `default-features` is specified that is no longer allowed in the 2024 Edition.

Unless the workspace specifies `default-features = false`, it is no longer allowed to specify `default-features = false` in an inherited package dependency. For example, with a workspace that specifies:

```
[workspace.dependencies]
regex = "1.10.4"
```

The following is now an error:

```
[package]
name = "foo"
version = "1.0.0"
edition = "2024"

[dependencies]
regex = { workspace = true, default-features = false } # ERROR
```

The reason for this change is to avoid confusion when specifying `default-features = false` when the default feature is already enabled, since it has no effect.

If you want the flexibility of deciding whether or not a dependency enables the default-features of a dependency, be sure to set `default-features = false` in the workspace definition. Just beware that if you build multiple workspace members at the same time, the features will be unified so that if one member sets `default-features = true` (which is the default if not explicitly set), the default-features will be enabled for all members using that dependency.

## Migration

When using `cargo fix --edition`, Cargo will automatically update your `Cargo.toml` file to remove `default-features = false` in this situation.

If you would prefer to update your `Cargo.toml` manually, check for any warnings when running a build and remove the corresponding entries. Previous editions should display something like:

```
warning: /home/project/Cargo.toml: `default-features` is ignored for regex,
since `default-features` was not specified for
`workspace.dependencies.regex`,
this could become a hard error in the future
```



# Rustdoc

The following chapters detail changes to Rustdoc in the 2024 Edition.

# Rustdoc combined tests

## Summary

- [Doctests](#) are now combined into a single binary which should result in a significant performance improvement.

## Details

Prior to the 2024 Edition, rustdoc's "test" mode would compile each code block in your documentation as a separate executable. Although this was relatively simple to implement, it resulted in a significant performance burden when there were a large number of documentation tests. Starting with the 2024 Edition, rustdoc will attempt to combine documentation tests into a single binary, significantly reducing the overhead for compiling doctests.

```
/// Adds two numbers
///
/// ```
/// assert_eq!(add(1, 1), 2);
/// ```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

/// Subtracts two numbers
///
/// ```
/// assert_eq!(subtract(2, 1), 1);
/// ```
pub fn subtract(left: u64, right: u64) -> u64 {
    left - right
}
```

In this example, the two doctests will now be compiled into a single executable. Rustdoc will essentially place each example in a separate function within a single binary. The tests still run in independent processes as they did before, so any global state (like global statics) should still continue to work correctly.<sup>1</sup>

This change is only available in the 2024 Edition to avoid potential incompatibilities with existing doctests which may not work in a combined executable. However, these incompatibilities are expected to be extremely rare.

<sup>1</sup> For more information on the details of how this work, see ["Doctests - How were they improved?"](#).

## **standalone\_crate** tag

In some situations it is not possible for rustdoc to combine examples into a single executable. Rustdoc will attempt to automatically detect if this is not possible. For example, a test will not be combined with others if it:

- Uses the `compile_fail` tag, which indicates that the example should fail to compile.
- Uses an `edition` tag, which indicates the edition of the example.<sup>2</sup>
- Uses global attributes, like the `global_allocator` attribute, which could potentially interfere with other tests.
- Defines any crate-wide attributes (like `#![feature(...)]`).
- Defines a macro that uses `$crate`, because the `$crate` path will not work correctly.

However, rustdoc is not able to automatically determine *all* situations where an example cannot be combined with other examples. In these situations, you can add the `standalone_crate` language tag to indicate that the example should be built as a separate executable. For example:

```
//! ```
//! let location = std::panic::Location::caller();
//! assert_eq!(location.line(), 5);
//! ```
```

This is sensitive to the code structure of how the example is compiled and won't work with the "combined" approach because the line numbers will shift depending on how the doctests are combined. In these situations, you can add the `standalone_crate` tag to force the example to be built separately just as it was in previous editions. E.g.:

```
//! ```standalone_crate
//! let location = std::panic::Location::caller();
//! assert_eq!(location.line(), 5);
//! ```
```

<sup>2</sup> Note that rustdoc will only combine tests if the entire crate is Edition 2024 or greater. Using the `edition2024` tag in older editions will not result in those tests being combined.

## Migration

There is no automatic migration to determine which doctests need to be annotated with the `standalone_crate` tag. It's very unlikely that any given doctest will not work correctly when migrated. We suggest that you update your crate to the 2024 Edition and then run your documentation tests and see if any fail. If one does, you will need to analyze whether it can be rewritten to be compatible with the combined approach, or alternatively, add the `standalone_crate` tag to retain the previous behavior.

Some things to watch out for and avoid are:

- Checking the values of `std::panic::Location` or things that make use of `Location`. The location of the code is now different since multiple tests are now located in the same test crate.
- Checking the value of `std::any::type_name`, which now has a different module path.

# Rustdoc nested include! change

## Summary

When a doctest is included with `include_str!`, if that doctest itself also uses `include!`, `include_str!`, or `include_bytes!`, the path is resolved relative to the Markdown file, rather than to the Rust source file.

## Details

Prior to the 2024 edition, adding documentation with `#[doc=include_str!("path/file.md")]` didn't carry span information into any doctests in that file. As a result, if the Markdown file was in a different directory than the source, any paths included had to be specified relative to the source file.

For example, consider a library crate with these files:

- `Cargo.toml`
- `README.md`
- `src/`
  - `lib.rs`
- `examples/`
  - `data.bin`

Let's say that `lib.rs` contains this:

```
#![doc=include_str!("../README.md")]
```

And assume this `README.md` file:

```
...
let _ = include_bytes!("../examples/data.bin");
//          ^^^ notice this
...
```

Prior to the 2024 edition, the path in `README.md` needed to be relative to the `lib.rs` file.

In 2024 and later, it is now relative to `README.md` itself, so we would update `README.md` to:

```
```\nlet _ = include_bytes!("examples/data.bin");\n```
```

## Migration

There is no automatic migration to convert the paths in affected doctests. If one of your doctests is affected, you'll see an error like this after migrating to the new edition when building your tests:

```
error: couldn't read `../examples/data.bin`: No such file or directory (os\nerror 2)\n--> src/../README.md:2:24\n  |\n2 | let _ = include_bytes!("../examples/data.bin");\n  |               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^\n    = note: this error originates in the macro `include_bytes` (in Nightly\nbuilds, run with -Z macro-backtrace for more info)\nhelp: there is a file with the same name in a different directory\n  |\n2 | let _ = include_bytes!("examples/data.bin");\n  |               ~~~~~~
```

To migrate your doctests to Rust 2024, update any affected paths to be relative to the file containing the doctests.

# Rustfmt

The following chapters detail changes to Rustfmt in the 2024 Edition.

# Rustfmt: Style edition

## Summary

User can now control which style edition to use with `rustfmt`.

## Details

The default formatting produced by Rustfmt is governed by the rules in the [Rust Style Guide](#).

Additionally, Rustfmt has a formatting stability guarantee that aims to avoid causing noisy formatting churn for users when updating a Rust toolchain. This stability guarantee essentially means that a newer version of Rustfmt cannot modify the *successfully formatted* output that was produced by a previous version of Rustfmt.

The combination of those two constraints had historically locked both the Style Guide and the default formatting behavior in Rustfmt. This impasse caused various challenges, such as preventing the ability to iterate on style improvements, and requiring Rustfmt to maintain legacy formatting quirks that were obviated long ago (e.g. nested tuple access).

[RFC 3338](#) resolved this impasse by establishing a mechanism for the Rust Style Guide to be aligned to Rust's Edition model wherein the Style Guide could evolve across Editions, and `rustfmt` would allow users to specify their desired Edition of the Style Guide, referred to as the Style Edition.

In the 2024 Edition, `rustfmt` now supports the ability for users to control the Style Edition used for formatting. The 2024 Edition of the Style Guide also includes enhancements to the Style Guide which are detailed elsewhere in this Edition Guide.

By default `rustfmt` will use the same Style Edition as the standard Rust Edition used for parsing, but the Style Edition can also be overridden and configured separately.

There are multiple ways to run `rustfmt` with the 2024 Style Edition:

With a `Cargo.toml` file that has `edition` set to `2024`, run:



```
cargo fmt
```

Or run `rustfmt` directly with `2024` for the edition to use the 2024 edition for both parsing and the 2024 edition of the Style Guide:

```
rustfmt lib.rs --edition 2024
```

The style edition can also be set in a `rustfmt.toml` configuration file:

```
style_edition = "2024"
```

Which is then used when running `rustfmt` directly:

```
rustfmt lib.rs
```

Alternatively, the style edition can be specified directly from `rustfmt` options:

```
rustfmt lib.rs --style-edition 2024
```

## Migration

Running `cargo fmt` or `rustfmt` with the 2024 edition or style edition will automatically migrate formatting over to the 2024 style edition formatting.

Projects who have contributors that may utilize their editor's format-on-save features are also strongly encouraged to add a `.rustfmt.toml` file to their project that includes the corresponding `style_edition` utilized within their project, or to encourage their users to ensure their local editor format-on-save feature is configured to use that same `style_edition`.

This is to ensure that the editor format-on-save output is consistent with the output when `cargo fmt` is manually executed by the developer, or the project's CI process (many editors will run `rustfmt` directly which by default uses the 2015 edition, whereas `cargo fmt` uses the edition specified in the `Cargo.toml` file)

# Rustfmt: Formatting fixes

## Summary

- Fixes to various formatting scenarios.

## Details

The 2024 style edition introduces several fixes to various formatting scenarios.

### **Don't align unrelated trailing comments after items or at the end of blocks**

Previously rustfmt would assume that a comment on a line following an item with a trailing comment should be indented to match the trailing comment. This has been changed so that those comments are not indented.

#### **Style edition 2021:**

```

pub const IFF_MULTICAST: ::c_int = 0x0000000800; // Supports multicast
                                                    // Multicast using broadcast.

add.

pub const SQ_CREATAB: u16 = 0x000e; // CREATE TABLE
pub const SQ_DRPTAB: u16 = 0x000f; // DROP TABLE
pub const SQ_CREIDX: u16 = 0x0010; // CREATE INDEX
                                                    //const SQ_DRPIDX: u16 = 0x0011; // DROP
INDEX
                                                    //const SQ_GRANT: u16 = 0x0012; // GRANT
                                                    //const SQ_REVOKE: u16 = 0x0013; // REVOKE

fn foo() {
    let f = bar(); // Donec consequat mi. Quisque vitae dolor. Integer
lobortis. Maecenas id nulla. Lorem.
                    // Id turpis. Nam posuere lectus vitae nibh. Etiam tortor
orci, sagittis
                    // malesuada, rhoncus quis, hendrerit eget, libero.
Quisque commodo nulla at
    let b = baz();

    let normalized = self.ctfont.all_traits().normalized_weight(); // [-1.0,
1.0]
                                                    //
TODO(emilio): It may make sense to make this range [.01, 10.0], to align
                                                    // with
css-fonts-4's range of [1, 1000].
}

```

## Style edition 2024:

```

pub const IFF_MULTICAST: ::c_int = 0x0000000800; // Supports multicast
// Multicast using broadcast. add.

pub const SQ_CREATAB: u16 = 0x000e; // CREATE TABLE
pub const SQ_DRPTAB: u16 = 0x000f; // DROP TABLE
pub const SQ_CREIDX: u16 = 0x0010; // CREATE INDEX
//const SQ_DRPIDX: u16 = 0x0011; // DROP INDEX
//const SQ_GRANT: u16 = 0x0012; // GRANT
//const SQ_REVOKE: u16 = 0x0013; // REVOKE

fn foo() {
    let f = bar(); // Donec consequat mi. Quisque vitae dolor. Integer
lobortis. Maecenas id nulla. Lorem.
    // Id turpis. Nam posuere lectus vitae nibh. Etiam tortor orci, sagittis
    // malesuada, rhoncus quis, hendrerit eget, libero. Quisque commodo nulla
at
    let b = baz();

    let normalized = self.ctfont.all_traits().normalized_weight(); // [-1.0,
1.0]
    // TODO(emilio): It may make sense to make this range [.01, 10.0], to
align
    // with css-fonts-4's range of [1, 1000].
}

```

## Don't indent strings in comments

Previously rustfmt would incorrectly attempt to format strings in comments.

### Original:

```

pub fn main() {
    /*    let s = String::from(
        "
hello
world
",
    ); */
}

```

### Style edition 2021:

```
pub fn main() {  
    /* let s = String::from(  
        "  
        hello  
        world  
        ",  
    ); */  
}
```

### Style edition 2024:

No change from original.

## Long strings don't prevent formatting expressions

In some situations, long strings would previously prevent the expression from being formatted.

### Style edition 2021:

```
fn main() {  
    let value = if x == "Excepteur sint occaecat cupidatat non proident, sunt  
in culpa qui officia deserunt mollit anim id est laborum." { 0 } else {10};  
  
    let x = Testing {  
        foo:  
        "long_long_long_long_long_long_long_lo_long_long_long_long_long__long_lo  
ng_long_long_long_long_",  
        bar:  
        "long_long_long_long_long_long_long_long_lo_long_long_lo long_long_l  
ong_lo_long_long_lo long_long_long_lo_long_long_lo",  
    };  
}
```

### Style edition 2024:

```
fn main() {
    let value = if x
        == "Excepteur sint occaecat cupidatat non proident, sunt in culpa qui
officia deserunt mollit anim id est laborum."
    {
        0
    } else {
        10
    };

    let x = Testing {
        foo:
"long_long_long_long_long_long_long_lo_long_long_long_long_long__long_lo
ng_long_long_long_long_",
        bar:
"long_long_long_long_long_long_long_long_lo_long_long_lo long_long_l
ong_lo_long_long_lo long_long_long_lo_long_long_lo",
    };
}
```

## Fixed indentation of generics in impl blocks

Generics in `impl` items had excessive indentation.

### Style edition 2021:

```
impl<
    Target: FromEvent<A> + FromEvent<B>,
    A: Widget2<Ctx = C>,
    B: Widget2<Ctx = C>,
    C: for<'a> CtxFamily<'a>,
> Widget2 for WidgetEventLifter<Target, A, B>
{
    type Ctx = C;
    type Event = Vec<Target>;
}
```

### Style edition 2024:

```
impl<
    Target: FromEvent<A> + FromEvent<B>,
    A: Widget2<Ctx = C>,
    B: Widget2<Ctx = C>,
    C: for<'a> CtxFamily<'a>,
> Widget2 for WidgetEventLifter<Target, A, B>
{
    type Ctx = C;
    type Event = Vec<Target>;
}
```

## Use correct indentation when formatting a complex fn

In some cases, a complex `fn` signature could end up with an unusual indentation that is now fixed.

### Style edition 2021:

```
fn build_sorted_static_get_entry_names(
    mut entries: Vec<(u8, &'static str)>,
) -> (impl Fn(
    AlphabeticalTraversal,
    Box<dyn dirents_sink::Sink<AlphabeticalTraversal>>,
) -> BoxFuture<'static, Result<Box<dyn dirents_sink::Sealed>, Status>>
    + Send
    + Sync
    + 'static) {
}
```

### Style edition 2024:

```
fn build_sorted_static_get_entry_names(
    mut entries: Vec<(u8, &'static str)>,
) -> (
    impl Fn(
        AlphabeticalTraversal,
        Box<dyn dirents_sink::Sink<AlphabeticalTraversal>>,
    ) -> BoxFuture<'static, Result<Box<dyn dirents_sink::Sealed>, Status>>
    + Send
    + Sync
    + 'static
) {
}
```

## Avoid extra space in nested tuple indexing expression

Nested tuple indexing expressions would incorrectly include an extra space.

### Style edition 2021:

```
fn main() {
    let _ = ((1,),).0 .0;
}
```

### Style edition 2024:

```
fn main() {
    let _ = ((1,),).0.0;
}
```

## End return/break/continue inside a block in a match with a semicolon

A `return`, `break`, or `continue` inside a block in a match arm was incorrectly missing a semicolon.

### Style edition 2021:

```
fn foo() {
    match 0 {
        0 => {
            return
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAA
        }
        _ => "",
    };
}
```

### Style edition 2024:



```
fn foo() {
    match 0 {
        0 => {
            return
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAA;
        }
        _ => "",
    };
}
```

## Long array and slice patterns are now wrapped

Long array and slice patterns were not getting wrapped properly.

### Style edition 2021:

```
fn main() {
    let [aaaaaaaaaaaaaaaaaaaaaaaaaa, bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb,
        cccccccccccccccccccccccccc, dddddddddddddddddddddddddd] =
        panic!();
}
```

### Style edition 2024:

```
fn main() {
    let [
        aaaaaaaaaaaaaaaaaaaaaaaaaa,
        bbbbbbbbbbbbbbbbbbbbbbbbbbbb,
        cccccccccccccccccccccccccc,
        dddddddddddddddddddddddd,
    ] = panic!();
}
```

## Format the last expression-statement as an expression

The last statement in a block which is an expression is now formatted as an expression.

### Style edition 2021:

```
fn main() {
    let toto = || {
        if true {
            42
        } else {
            24
        }
    };

    {
        T
    }
}
```

### Style edition 2024:

```
fn main() {
    let toto = || {
        if true { 42 } else { 24 }
    };

    { T }
}
```

## Same formatting between function and macro calls

Some formatting is now the same in a macro invocation as it is in a function call.

### Style edition 2021:

```
fn main() {
    macro_call!(HAYSTACK
        .par_iter()
        .find_any(|&&x| x[0] % 1000 == 999)
        .is_some());

    fn_call(
        HAYSTACK
        .par_iter()
        .find_any(|&&x| x[0] % 1000 == 999)
        .is_some(),
    );
}
```

### Style edition 2024:

```
fn main() {
    macro_call!(
        HAYSTACK
        .par_iter()
        .find_any(|&&x| x[0] % 1000 == 999)
        .is_some()
    );

    fn_call(
        HAYSTACK
        .par_iter()
        .find_any(|&&x| x[0] % 1000 == 999)
        .is_some(),
    );
}
```

## Force block closures for closures with a single loop body

Closures with a single loop are now formatted as a block expression.

### Style edition 2021:

```
fn main() {
    thread::spawn(|| loop {
        println!("iteration");
    });
}
```

### Style edition 2024:

```
fn main() {
    thread::spawn(|| {
        loop {
            println!("iteration");
        }
    });
}
```

## Empty lines in where clauses are now removed

Empty lines in a `where` clause are now removed.

## Style edition 2021:

```
fn foo<T>(_: T)
where
    T: std::fmt::Debug,

    T: std::fmt::Display,
{
}
```

## Style edition 2024:

```
fn foo<T>(_: T)
where
    T: std::fmt::Debug,
    T: std::fmt::Display,
{
}
```

## Fixed formatting of a let-else statement with an attribute

If a let-else statement had an attribute, then it would cause the `else` clause to incorrectly wrap the `else` part separately.

## Style edition 2021:

```
fn main() {
    #[cfg(target_os = "linux")]
    let x = 42
    else {
        todo!()
    };

    // This is the same without an attribute.
    let x = 42 else { todo!() };
}
```

## Style edition 2024:

```
fn main() {  
    #[cfg(target_os = "linux")]  
    let x = 42 else { todo!() };  
  
    // This is the same without an attribute.  
    let x = 42 else { todo!() };  
}
```

## Off-by-one error for wrapping enum variant doc comments

When using the `wrap_comments` feature, the comments were being wrapped at a column width off-by-one.

### Original:

```
pub enum Severity {  
    /// But here, this comment is 120 columns wide and the formatter wants to  
    split it up onto two separate lines still.  
    Error,  
    /// This comment is 119 columns wide and works perfectly. Lorem ipsum.  
    lorem ipsum. lorem ipsum. lorem ipsum lorem.  
    Warning,  
}
```

### Style edition 2021:

```
pub enum Severity {  
    /// But here, this comment is 120 columns wide and the formatter wants to  
    split it up onto two separate lines  
    /// still.  
    Error,  
    /// This comment is 119 columns wide and works perfectly. Lorem ipsum.  
    lorem ipsum. lorem ipsum. lorem ipsum lorem.  
    Warning,  
}
```

### Style edition 2024:

```
pub enum Severity {
    /// But here, this comment is 120 columns wide and the formatter wants to
split it up onto two separate lines still.
    Error,
    /// This comment is 119 columns wide and works perfectly. Lorem ipsum.
lorem ipsum. lorem ipsum. lorem ipsum lorem.
    Warning,
}
```

## Off-by-one error for `format_macro_matchers`

When using the `format_macro_matchers` feature, the matcher was being wrapped at a column width off-by-one.

### Style edition 2021:

```
macro_rules! test {
    ($aasdfghj:expr, $qwertyuiop:expr, $zxcvbnmasdfghjkl:expr,
    $aeiouaeiouaeio:expr, $add:expr) => {{
        return;
    }};
}
```

### Style edition 2024:

```
macro_rules! test {
    (
        $aasdfghj:expr, $qwertyuiop:expr, $zxcvbnmasdfghjkl:expr,
        $aeiouaeiouaeio:expr, $add:expr
    ) => {{
        return;
    }};
}
```

## Fixed failure with `=>` in comment after match `=>`

In certain circumstances if a comment contained a `=>` after the `=>` in a match expression, this would cause a failure to format correctly.

### Style edition 2021:

```
fn main() {  
    match a {  
        _ =>  
            // comment with =>  
            {  
                println!("A")  
            }  
    }  
}
```

### Style edition 2024:

```
fn main() {  
    match a {  
        _ =>  
            // comment with =>  
            {  
                println!("A")  
            }  
    }  
}
```

## Multiple inner attributes in a match expression indented incorrectly

Multiple inner attributes in a match expression were being indented incorrectly.

### Style edition 2021:

```
pub fn main() {  
    match a {  
        #![attr1]  
        #![attr2]  
        _ => None,  
    }  
}
```

### Style edition 2024:

```
pub fn main() {  
    match a {  
        #![attr1]  
        #![attr2]  
        _ => None,  
    }  
}
```

## Migration

The change can be applied automatically by running `cargo fmt` or `rustfmt` with the 2024 Edition. See the [Style edition](#) chapter for more information on migrating and how style editions work.



# Rustfmt: Raw identifier sorting

## Summary

`rustfmt` now properly sorts [raw identifiers](#).

## Details

The [Rust Style Guide](#) includes [rules for sorting](#) that `rustfmt` applies in various contexts, such as on imports.

Prior to the 2024 Edition, when sorting `rustfmt` would use the leading `r#` token instead of the ident which led to unwanted results. For example:

```
use websocket::client::ClientBuilder;
use websocket::r#async::futures::Stream;
use websocket::result::WebSocketError;
```

In the 2024 Edition, `rustfmt` now produces:

```
use websocket::r#async::futures::Stream;
use websocket::client::ClientBuilder;
use websocket::result::WebSocketError;
```

## Migration

The change can be applied automatically by running `cargo fmt` or `rustfmt` with the 2024 Edition. See the [Style edition](#) chapter for more information on migrating and how style editions work.

# Rustfmt: Version sorting

## Summary

`rustfmt` utilizes a new sorting algorithm.

## Details

The [Rust Style Guide](#) includes [rules for sorting](#) that `rustfmt` applies in various contexts, such as on imports.

Previous versions of the Style Guide and Rustfmt generally used an "ASCIIbetical" based approach. In the 2024 Edition this is changed to use a version-sort like algorithm that compares Unicode characters lexicographically and provides better results in ASCII digit comparisons.

For example with a given (unsorted) input:

```
use std::num::{NonZeroU32, NonZeroU16, NonZeroU8, NonZeroU64};
use std::io::{Write, Read, stdout, self};
```

In the prior Editions, `rustfmt` would have produced:

```
use std::io::{self, stdout, Read, Write};
use std::num::{NonZeroU16, NonZeroU32, NonZeroU64, NonZeroU8};
```

In the 2024 Edition, `rustfmt` now produces:

```
use std::io::{self, Read, Write, stdout};
use std::num::{NonZeroU8, NonZeroU16, NonZeroU32, NonZeroU64};
```

## Migration

The change can be applied automatically by running `cargo fmt` or `rustfmt` with the 2024 Edition. See the [Style edition](#) chapter for more information on migrating and how style editions work.